

Chapter 10

Dolby AC-3 Audio

The Dolby AC-3 standard describes an algorithm suitable for coding an audio signal with channel formats ranging from monophonic to 5.1 channels into a serial bit stream with data rates ranging from 32 to 640 kbits/s. The AC-3 algorithm is based on the transform coding technology developed for the AC-2 coding algorithm in 1989. The AC-2 algorithm operated on stereo signals only and relied on a 4-2-4 multichannel matrix system to transmit a four-channel surround-sound signal. By 1990 it was realized that the data compression performed by the 4-2-4 matrix system could be better performed as part of the coding algorithm and the development of a multichannel audio coder was initiated. The first implementation of such a multichannel coder was used for providing a 5.1 channel digital soundtrack in cinemas in 1991. This early implementation of the AC-3 algorithm (or Dolby digital as it is usually called) converted a 5.1 channel input signal into a serial bit stream with a single output bit rate of 320 kbits/s. This rate was determined by the maximum amount of data that could be reliably placed and extracted from the area between the sprocket holes on one side of the 35 mm film.

In February 1992, the U.S. Advanced Television Systems Committee formally recommended the use of a 5.1 channel audio signal for the U.S. HDTV service. The AC-3 coder was considered as a possible technique to provide this audio signal. However, the requirements placed for an audio coder providing such a service are more diverse than for a coder intended solely for digital cinema applications. So the basic AC-3 algorithm was improved to provide extra functionality. These extra features included support for a range of output bit rates, the ability to downmix the 5.1 channel output to fewer channels, and the ability to reproduce the output signal with a restricted dynamic range. In late 1993, the AC-3 coding algorithm was formally evaluated using subjective testing and was subsequently recommended for use in the U.S. HDTV system. The standard was initially published in November 1994 and a revised edition containing a backwards-compatible alternate bit-stream syntax was published in August 2001 [1]. (The latest version of the standard can be found at www.atsc.org.)

A block diagram of the AC-3 encoder is shown in Figure 10.1. The first step in the encoding process is to convert a sequence of audio input samples into a block of

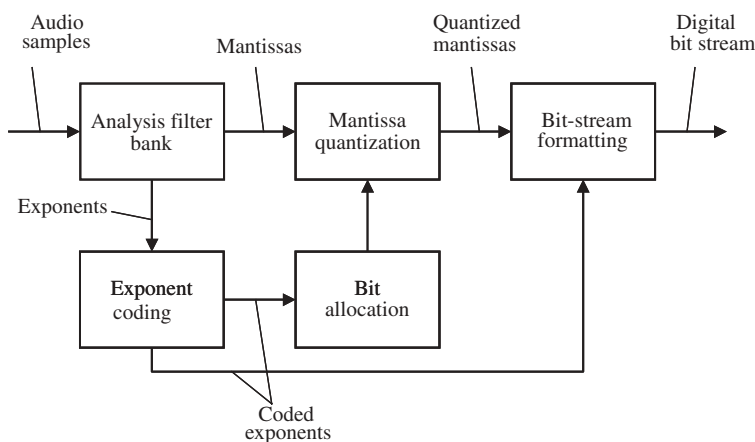


Figure 10.1 Block diagram of the AC-3 encoder. © Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

frequency coefficients. This is achieved using a time-domain aliasing cancellation (TDAC) transform approach as discussed in Section 8.6 of Chapter 8. Then each frequency coefficient in the block is represented in floating point format as an exponent and a mantissa. The exponent acts as a scalefactor for the mantissa and indicates the number of leading zeros in the binary integer representation of the coefficient. Exponent values can range from 0 to 24 and are fixed at 24 if the coefficient has more leading zeros. The set of exponents is also used as an approximation of the power spectral density of the signal and is referred to as the *spectral envelope* of the signal. This spectral envelope is then used in the bit-allocation process to determine the quantizer step size for the mantissas for each coefficient. The bit-allocation process uses a model of frequency masking to determine the precision required for each mantissa.

An AC-3 bit stream consists of a sequence of synchronization frames as shown in Figure 10.2. Each synchronization frame includes six *audio blocks*, each of which contains the coded data representing 256 new input samples. Each synchronization frame begins with a synchronization information (SI) header, which contains information necessary to acquire and maintain synchronization, and a bit-stream information (BSI) header that contains parameters describing the coded audio service. The synchronization frame ends with an error check field that contains a CRC code word used for error detection, and an auxiliary (Aux) data field may also be included after the coded audio blocks.

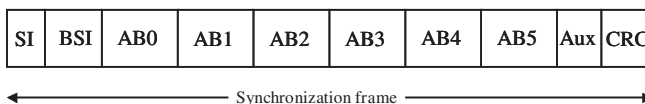


Figure 10.2 Format of an AC-3 synchronization frame.

The algorithms and bit-stream syntax associated with a basic implementation of the AC-3 encoder and decoder are described in more detail in the following sections.

10.1. ENCODER

A more detailed flow diagram of the encoding process is shown in Figure 10.3, and the individual components of this diagram are explained in more detail in the following section.

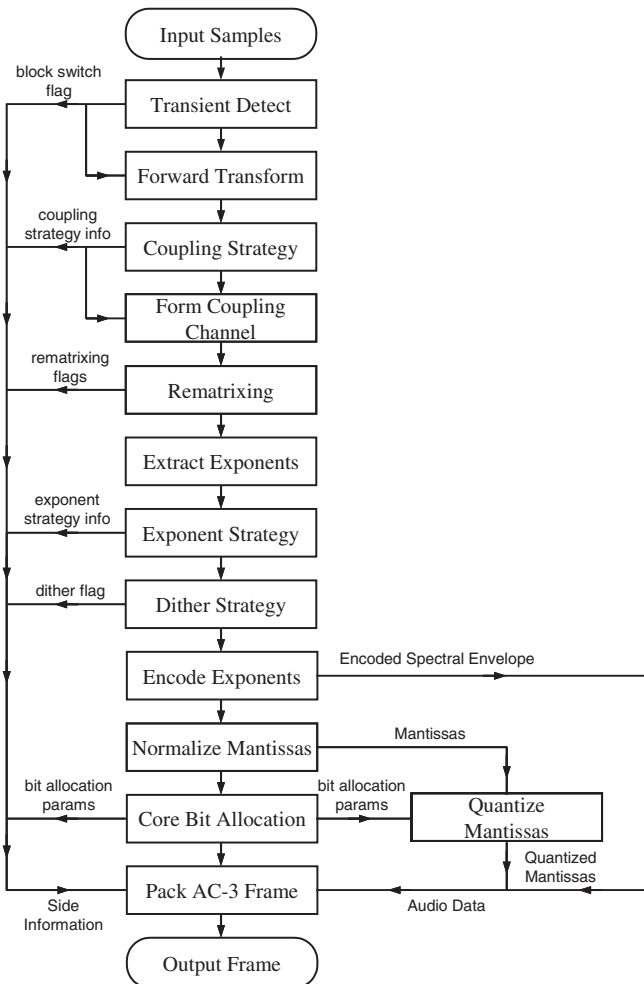


Figure 10.3 Flow diagram of the AC-3 encoding process.

Table 10.1 Sample rate code words.

fs_{cod}	Sampling rate (kHz)
“00”	48
“01”	44.1
“10”	32
“11”	Reserved

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.1.1. Audio Input Format

The AC-3 encoder accepts audio samples with a precision of up to 24 bits. The sampling rate of the input signal is directly proportional to the output. The input sample rate must be chosen such that each AC-3 audio frame consists of the audio data for 1536 audio samples. If the input audio data is sampled at a rate other than that required, sample rate conversion must be preformed to achieve the desired input sample rate. The input sample rate is indicated by the 2-bit code word **fs_{cod}**. The sampling rate indicated by each value of this code word is shown in Table 10.1.

The input audio signal may have one of the eight possible channel configurations (usually referred to as *audio coding modes*) as shown in Table 10.2. The *p/q* notation is typically used in multichannel systems to indicate *p* front and *q* back channels. These channels are referred to as *full-bandwidth* channels as opposed to the low-frequency effects channel that typically has a much smaller bandwidth. So if the low-frequency effects channel is present, the total number of channels to be transmitted is equal to the number of full-bandwidth channels plus one. The 1+1 notation is used to indicate two completely independent input channels.

Individual input channels may also be filtered to remove the DC components. This allows more efficient coding of the input signal. A typical encoder would filter the

Table 10.2 Audio coding modes.

ac_{mod}	Audio coding mode	nfchans	Channel array ordering
“000”	1+1	2	Ch1, Ch2
“001”	1/0	1	C
“010”	2/0	2	L, R
“011”	3/0	3	L, C, R
“100”	2/1	3	L, R, S
“101”	3/1	4	L, C, R, S
“110”	2/2	4	L, R, Ls, Rs
“111”	3/2	5	L, C, R, Ls, Rs

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

input signals using a high-pass filter with a cutoff frequency of 3 Hz. The LFE channel may also be filtered using a low-pass filter with a cutoff frequency of 120 Hz.

10.1.2. Transient Detection

One common problem that is encountered by most digital audio encoders is the occurrence of *preecho*. This problem is associated with transients in the audio signal that take the form of a period of silence followed by the rapid onset of a loud percussive sound such as a cymbal clash or a drum beat. If these transients occur in the middle of an audio block, the quantization strategy for that block is determined based on the loud sound rather than the period of silence, and this generally results in relatively large quantizer step sizes being chosen for the entire block. Because the frequency-domain coefficients translate back to sinusoidal waveforms that extend over the entire reconstructed audio signal, the large amount of distortion associated with the coarse quantization strategy is present in the period of silence at the beginning of the block and is not hidden by either frequency masking or backward temporal masking. Hence, the noise at the beginning of the block is usually clearly audible and is referred to as *preecho*.

One solution to this problem is to use a shorter window length for the transform so that backward temporal masking can be used to mask the *preechoes*. However, shorter window lengths result in an increase in block overhead information and an increase in the overlap of subbands in the frequency domain. Therefore, a typical approach in audio coding is to switch between long and short transform windows depending on the transient behavior of the input audio signal.

In the AC-3 encoder, transient detection is used to determine when to switch between a single 512-sample window and two 256-sample windows. The transient detection algorithm is divided into four basic steps:

1. high-pass filtering,
2. segmentation of the block into subblocks,
3. peak amplitude detection within each subblock, and
4. threshold comparison.

The high-pass filter used in the first step should have a cutoff frequency of 8 kHz. Each block of 256 high-pass filtered samples is then segmented into a hierarchical tree of subblocks. Level 1 in the tree corresponds to the entire 256-sample block, Level 2 consists of two 128-sample subblocks, and Level 3 consists of four 64-sample subblocks. The peak detection algorithm then determines the sample with the largest magnitude in each subblock in each level of the hierarchical tree. The peak magnitude in subblock k of Level j is denoted by $P_{j,k}$.

The first step in the threshold comparison process is to determine if the maximum signal level in the block is above a *silence threshold*. This is achieved by comparing $P_{1,1}$ with the silence threshold value of 100/32768. If the value of $P_{1,1}$ is below the silence threshold, the remaining threshold comparison is not performed and a transient is not detected in the current 256-sample block. If $P_{1,1}$ is above the silence

Table 10.3 Threshold values of the transient detection algorithm.

Level (j)	T_j
1	0.1
2	0.075
3	0.05

threshold, the next step in the threshold comparison process is to compare the ratio of the peak values of adjacent subblocks at each level with a predefined threshold T_j . The threshold values for each level are shown in Table 10.3.

If any of these ratios is above the threshold, a transient is detected in the current 256-sample block. Note that for the first subblock of each level, the ratio of peak values is calculated using the last subblock of the same level from the previously calculated tree. For example, $P_{3,1}$ of the current tree and $P_{3,4}$ of the preceding tree would be used to calculate the ratio for the first subblock in Level 3 of the current tree.

If a transient is detected in the second half of the 512-sample block, this is indicated by a value of 1 for the **blksw[ch]** flag, and the following forward transform operation is performed using two 256-point transform operations.

10.1.3. Forward Transform

The TDAC transform process requires that the input audio samples be multiplied by a window function. The values for the first half of this window function are given in Table 10.4. These values are mirrored and repeated to form a 512-point symmetrical window function.

The forward transform is then performed using the following equation:

$$X(k) = \frac{-2}{N} \sum_{n=0}^{N-1} w(n)x(n) \cos\left(\frac{2\pi}{4N}(2n+1)(2k+1) + \frac{\pi}{4}(2k+1)(1+\alpha)\right) \\ \text{for } k = 0, \dots, N/2 - 1 \quad (10.1)$$

Where $x(n)$ is the set of input samples, $w(n)$ is the window function with the values shown in Table 10.4, and $X(k)$ is the set of transform coefficients. The variable α is used to control the time offset for the transform basis functions and is given by

$$\alpha = \begin{cases} -1, & \text{for the first short transform} \\ 0, & \text{for the long transform} \\ +1, & \text{for the second short transform} \end{cases} \quad (10.2)$$

Figure 10.4 shows an example of a set of 512 samples of a typical audio waveform, and Figure 10.5 shows the set of 256 output coefficients for these input samples when the forward transform is applied using a single long window.

Table 10.4 Window function for the TDAC transform.

	$w(i + j)$									
	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$	$j = 8$	$j = 9$
$i = 0$	0.00014	0.00024	0.00037	0.00051	0.00067	0.00086	0.00107	0.00130	0.00157	0.00187
$i = 10$	0.00220	0.00256	0.00297	0.00341	0.00390	0.00443	0.00501	0.00564	0.00632	0.00706
$i = 20$	0.00785	0.00871	0.00962	0.01061	0.01166	0.01279	0.01399	0.01526	0.01662	0.01806
$i = 30$	0.01959	0.02121	0.02292	0.02472	0.02662	0.02863	0.03073	0.03294	0.03527	0.03770
$i = 40$	0.04025	0.04292	0.04571	0.04862	0.05165	0.05481	0.05810	0.06153	0.06508	0.06878
$i = 50$	0.07261	0.07658	0.08069	0.08495	0.08935	0.09389	0.09859	0.10343	0.10842	0.11356
$i = 60$	0.11885	0.12429	0.12988	0.13563	0.14152	0.14757	0.15376	0.16011	0.16661	0.17325
$i = 70$	0.18005	0.18699	0.19407	0.20130	0.20867	0.21618	0.22382	0.23161	0.23952	0.24757
$i = 80$	0.25574	0.26404	0.27246	0.28100	0.28965	0.29841	0.30729	0.31626	0.32533	0.33450
$i = 90$	0.34376	0.35311	0.36253	0.37204	0.38161	0.39126	0.40096	0.41072	0.42054	0.43040
$i = 100$	0.44030	0.45023	0.46020	0.47019	0.48020	0.49022	0.50025	0.51028	0.52031	0.53033
$i = 110$	0.54033	0.55031	0.56026	0.57019	0.58007	0.58991	0.59970	0.60944	0.61912	0.62873
$i = 120$	0.63827	0.64774	0.65713	0.66643	0.67564	0.68476	0.69377	0.70269	0.71150	0.72019
$i = 130$	0.72877	0.73723	0.74557	0.75378	0.76186	0.76981	0.77762	0.78530	0.79283	0.80022
$i = 140$	0.80747	0.81457	0.82151	0.82831	0.83496	0.84145	0.84779	0.85398	0.86001	0.86588
$i = 150$	0.87160	0.87716	0.88257	0.88782	0.89291	0.89785	0.90264	0.90728	0.91176	0.91610
$i = 160$	0.92028	0.92432	0.92822	0.93197	0.93558	0.93906	0.94240	0.94560	0.94867	0.95162
$i = 170$	0.95444	0.95713	0.95971	0.96217	0.96451	0.96674	0.96887	0.97089	0.97281	0.97463
$i = 180$	0.97635	0.97799	0.97953	0.98099	0.98236	0.98366	0.98488	0.98602	0.98710	0.98811
$i = 190$	0.98905	0.98994	0.99076	0.99153	0.99225	0.99291	0.99353	0.99411	0.99464	0.99513
$i = 200$	0.99558	0.99600	0.99639	0.99674	0.99706	0.99736	0.99763	0.99788	0.99811	0.99831
$i = 210$	0.99850	0.99867	0.99882	0.99895	0.99908	0.99919	0.99929	0.99938	0.99946	0.99953
$i = 220$	0.99959	0.99965	0.99969	0.99974	0.99978	0.99981	0.99984	0.99986	0.99988	0.99990
$i = 230$	0.99992	0.99993	0.99994	0.99995	0.99996	0.99997	0.99998	0.99998	0.99998	0.99999
$i = 240$	0.99999	0.99999	0.99999	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000
$i = 250$	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000				

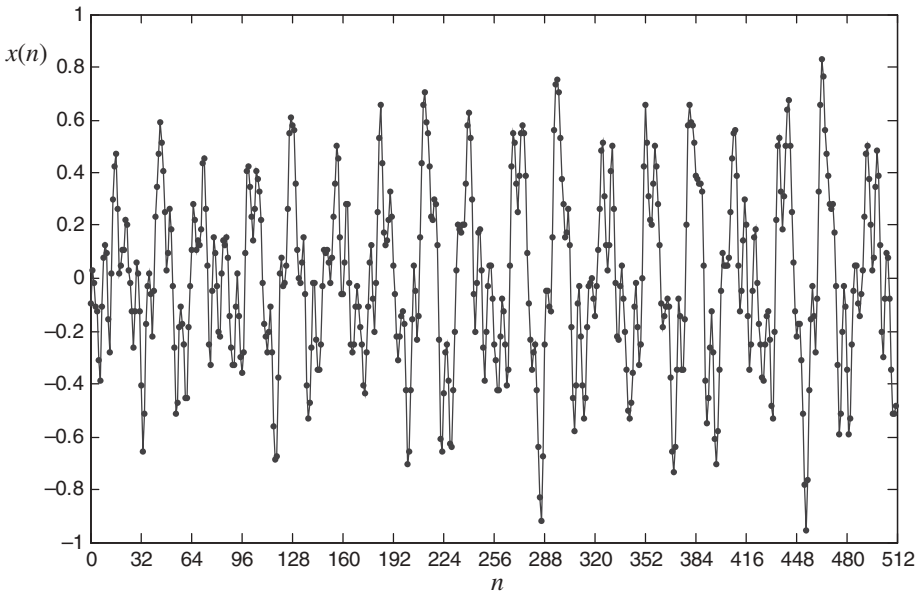


Figure 10.4 An example of a set of 512 input samples of an audio waveform.

Each frequency coefficient is then converted to an exponent and a mantissa. The exponent indicates the number of leading zeros in the binary integer representation of the coefficient. The exponents are integer values ranging from 0 to 24 and are fixed at 24 if the coefficient has more leading zeros. For example, a coefficient with a value of 0.1 would be represented as 0.8×2^{-3} , where 0.8 is the mantissa value and

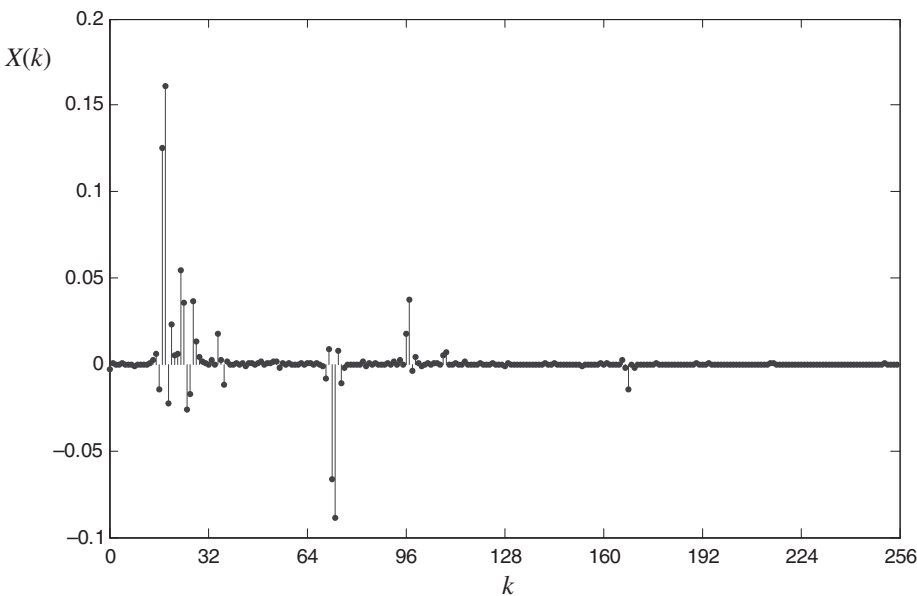


Figure 10.5 Output coefficients of the forward transform for the input samples shown in Figure 10.4.

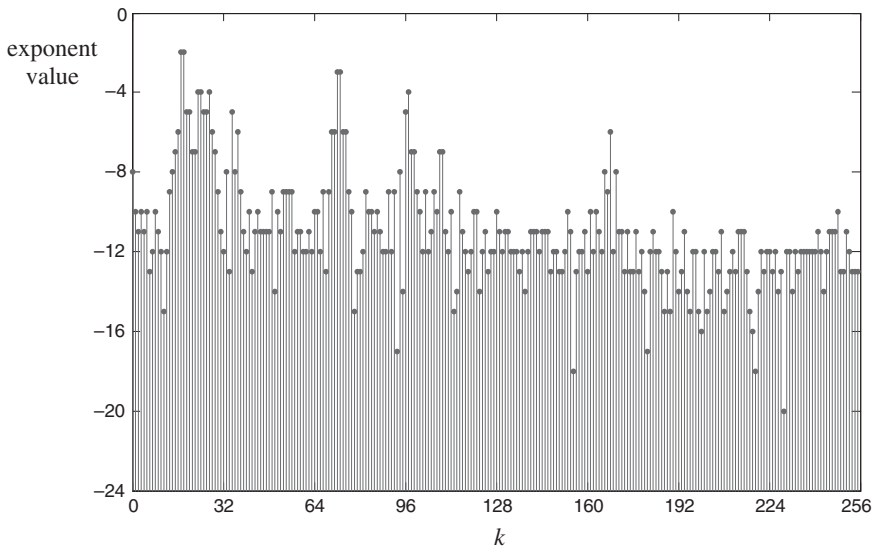


Figure 10.6 Exponent values for the output coefficients shown in Figure 10.5.

3 is the exponent value. Figure 10.6 shows the exponent values for the output coefficients shown in Figure 10.5. Note that the exponent values are always negative, so only the magnitude is required to be transmitted to the decoder. The method for coding the exponent and mantissa values is discussed in Sections 10.1.7 and 10.1.9.

10.1.4. Channel Coupling

For multichannel audio signals, there is often a significant amount of correlation between the different channels in the audio signal. *Channel coupling* is a means of exploiting this correlation to reduce the output bit rate of the AC-3 coder. The basis of the technique is to combine the high-frequency portion of several channels to form a single *coupling channel*. The coefficients of this coupling channel are transmitted as a set of mantissas and exponents in the same manner as the uncoupled coefficients of each channel. Then, in order to reconstruct an approximation of the original channels at the decoder, the ratio of the magnitude of the original coefficients in each channel to the coefficients in the coupling channel is also calculated. This ratio is called a *coupling coordinate* and is not calculated for each coefficient but is instead calculated for groups of coefficients called *coupling bands*.

The justification for reducing the bit rate in this way stems from the way the human ear perceives the directionality of an audio source. At frequencies above about 2 kHz, the ear determines the direction of a source using the level difference and interaural time delay of the envelope of the signal. So, for multichannel signals, as long as approximately the same signal envelope for the high-frequency components is maintained, the ear still perceives the correct directional information from the decoded sound source.

Table 10.5 Coupling subbands.

Subband No.	Start coefficient	End coefficient
0	37	48
1	49	60
2	61	72
3	73	84
4	85	96
5	97	108
6	109	120
7	121	132
8	133	144
9	145	156
10	157	168
11	169	180
12	181	192
13	193	204
14	205	216
15	217	228
16	229	240
17	241	252

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Coupling may be performed on frequency coefficients 37 to 252 only. These coefficients are grouped into 18 *coupling subbands*, with 12 coefficients in each subband. The coefficient numbers corresponding to the beginning and end of each coupling subband are given in Table 10.5.

The parameters **cplbegf** and **cplendf** indicate the first and last coupling subbands, respectively, that are included in the coupling process. The parameter **cplendf** is transmitted as a 4-bit unsigned integer and, hence, has a maximum value of 15, so the decoder adds 2 to the value of **cplendf** to obtain the last coupling band used.

The coupling subbands may be combined to form coupling bands (coupling coordinates are transmitted for each coupling band in each channel included in the coupling process). The parameter **cplbndstre[sbnd]** is used to indicate which subbands have been combined and which subbands are treated independently. **cplbndstre[sbnd]** is treated as an array of single bit elements with one element for each subband included in the coupling process, except for the first subband. No element is transmitted in **cplbndstre[sbnd]** for the first subband included in the coupling process. A value of 1 for a bit in **cplbndstre[sbnd]** indicates that the corresponding subband should be combined with the subband immediately below it in frequency, and a value of 0 indicates that the subband should start a new coupling band. For example, a basic encoder coupling strategy would produce the following coupling parameters:

```

cplbegf = 6
cplendf = 12
cplbndstrc = 00110111

```

These parameters indicate that the coupling is performed using subbands 6 to 14 and that four coupling bands are formed from subbands 6, 7, 8–10, and 11–14.

The coefficients of the coupling channel are calculated by averaging the coefficients of the individual channels included in the coupling process. Only those coefficients that correspond to the coupling bands are included in this process. However, simply averaging the coefficients may lead to an inaccurate estimate of the combined signal power when the coefficients to be combined represent signals that are close to 180° out-of-phase with each other. If this is the case, then the coefficients in the out-of-phase bands may be negated before the coupling channel is determined. This does not usually result in any perceived difference in the directionality of the sound source since the human auditory system does not use the interaural phase shift to determine directionality at these frequencies.

For the special case of 2/0 mode (left and right side channels only) it is possible to restore the relative phase of the two channels at the decoder. This is achieved by sending phase restoration information to the decoder using the parameter **phsflg[bnd]**. This parameter is treated as an array of single bit elements with one element for each coupling band. A value of 1 for a bit in **phsflg[bnd]** indicates that the coefficients of the right channel in the corresponding coupling band have been negated before the coupling channel was formed.

EXAMPLE 10.1

Figure 10.7 shows an example of the left and right audio signals for a stereo pair. Note that the amplitude of the high-frequency components is much larger in the right-hand signal than in the left.

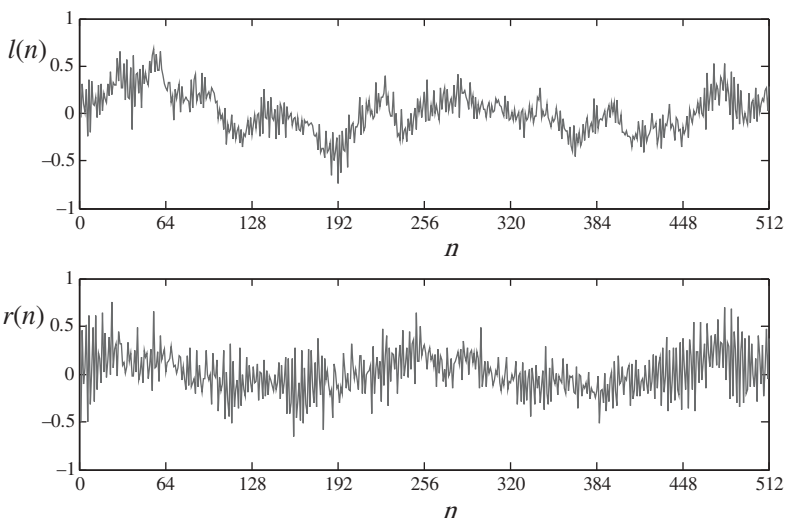


Figure 10.7 Audio signals for the left and right side channels of a stereo pair.

If coupling is to be performed on these two channels using subbands 6 to 14, Figure 10.8 shows the frequency coefficients that would be used for the two channels.

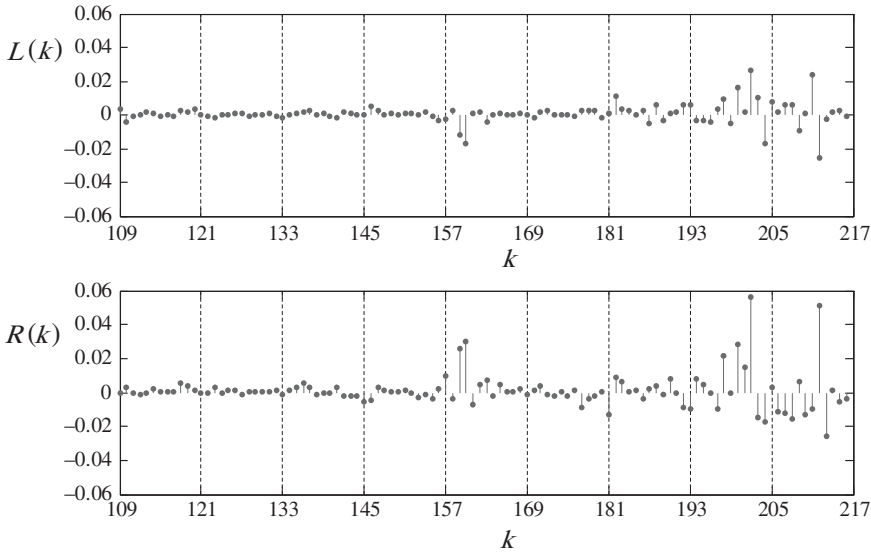


Figure 10.8 Frequency coefficients corresponding to subbands 6 to 14 for the left and right channels shown in Figure 10.7.

The next step in the coupling process is to adjust the phase of the right channel to avoid phase cancellation and combine subbands into bands. One suitable coupling strategy for these channels would be to form six coupling bands from subbands 6, 7, 8, 9–11, 12–13 and 14. For this strategy, the coupling parameters to be transmitted to the decoder are

cplbegf = 6
cplendf = 12
cplbndstrc = 00011010
phsflg = 010101

Now the coupling channel can be formed by averaging the coefficients from the left channel and the phase-adjusted right channel. The resulting coupling channel coefficients are shown in Figure 10.9.

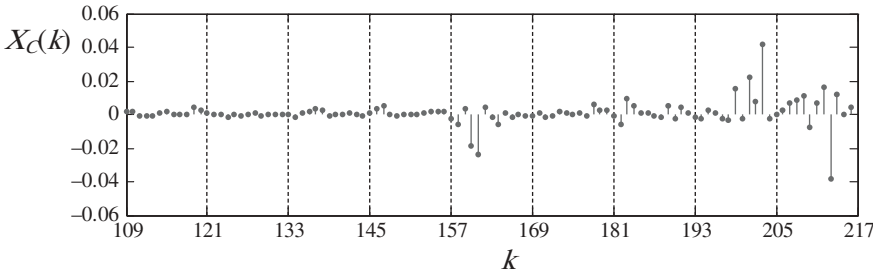


Figure 10.9 Coupling channel coefficients. ■

The next step in the coupling process is to calculate coupling coordinates for each band in each coupled channel. These coupling coordinates are determined by dividing the average magnitude of the original coefficients in a coupled band by the average magnitude of the coefficients in the same band of the coupling channel. The coupling coordinates for each band are transmitted using a 4-bit mantissa **cpcomant[ch][bnd]** and a 4-bit exponent **cpcoexp[ch][bnd]**.

To guarantee that the coupling coordinate values used to generate the mantissa and exponent are always in the range 0.0 to 1.0, coupling coordinate values should be limited to the range 0.0 to 8.0. Then, before converting the coordinate values to a floating-point representation, they are scaled to the range 0.0 to 1.0 by dividing by a factor of 8.0.

The coupling exponents indicate the number of leading zeros in the binary representation of the fractional coupling coordinates. Except for the case when the exponent value is 15, the mantissa values are always positive values in the range 0.5–1.0. Hence when the exponent value is less than 15, the most significant bit of the mantissa is always 1 and is not transmitted. So when the exponent value is less than 15, the 4-bit value of **cpcomant[ch][bnd]** is generated by multiplying the mantissa value by 32 and then subtracting 16 from the result. If the exponent value is equal to 15, **cpcomant[ch][bnd]** is calculated by multiplying the mantissa value by 16.

The dynamic range of the coupling coordinates can be increased by transmitting a 2-bit master coupling coordinate **mstreplco[ch]** for any of the coupled channels. The exponent values for all coupling coordinates in the corresponding channel are increased three times the value of **mstreplco[ch]**.

EXAMPLE 10.2

Figure 10.10 shows the frequency coefficient values and the average coefficient magnitudes for each coupling band of the left and right side channels shown in Figure 10.7, and Figure 10.11

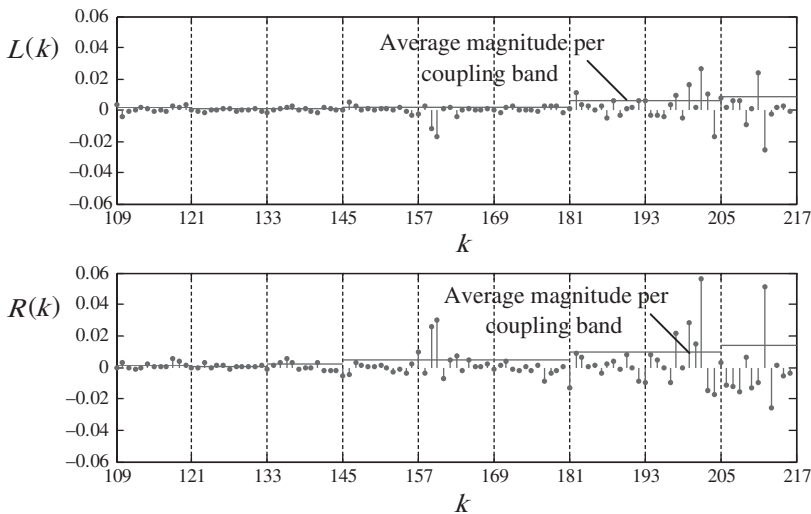


Figure 10.10 Frequency coefficient values and the average coefficient magnitudes for each coupling band of the left and right side channels shown in Figure 10.7.

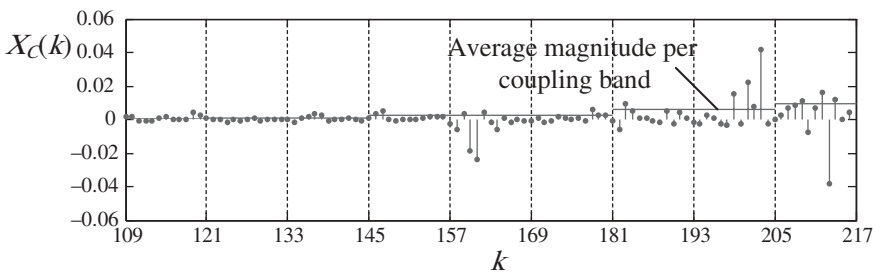


Figure 10.11 Frequency coefficient values and the average coefficient magnitudes for each band of the coupling channel.

shows the average coefficient magnitudes for each band of the coupling channel generated in the previous example.

Now, using these average magnitude values, the coupling coordinates for the six bands in each channel are generated. The values of the average coefficient magnitudes for each band are shown in Table 10.6 along with the corresponding coupling coordinates for each band of the left and right side channels.

Table 10.6 Average coefficient magnitudes and the corresponding coupling coordinates for each coupling band.

Coupling band	Average magnitude ($\times 10^{-3}$)			Coupling coordinates	
	Left	Right	Coupling	Left	Right
0	1.68	1.60	1.28	1.3131	1.2522
1	0.80	0.82	0.54	1.4803	1.5296
2	1.02	2.00	1.09	0.9353	1.8273
3	2.08	4.36	2.89	0.7216	1.5112
4	5.58	9.44	5.94	0.9393	1.5908
5	8.65	14.34	9.52	0.9078	1.5056

Then, after converting these coupling coordinates into a floating point representation, the values to be transmitted for **cplcomant[ch][bnd]** and **cplcoexp[ch][bnd]** are shown in

Table 10.7 Transmitted values for **cplcomant[ch][bnd]** and **cplcoexp[ch][bnd]** and the corresponding decoded coupling coordinates.

cplcomant		cplcoexp		Decoded coordinates	
Left	Right	Left	Right	Left	Right
5	4	2	2	1.3125	1.25
7	8	2	2	1.4375	1.5
13	13	3	2	0.90625	1.8125
7	8	3	2	0.71875	1.5
14	9	3	2	0.9375	1.5625
13	8	3	2	0.90625	1.5

Table 10.7 along with corresponding values for the coupling coordinates that would be generated at the decoder.

At the decoder, the values of each element of **phsflg[bnd]** and the decoded coupling coordinates would be used to reconstruct the high-frequency portions of the original left and right side channels. Figure 10.12 shows the original and reconstructed frequency coefficients that would be obtained for the left and right side channels shown in Figure 10.7. Figure 10.13 shows the reconstructed audio signals obtained by performing the inverse TDAC transform on the original low-frequency coefficients and the reconstructed high-frequency coefficients. Figures 10.7 and 10.13 show that the coupling process has maintained the relationship between the high-frequency envelopes of the two channels.

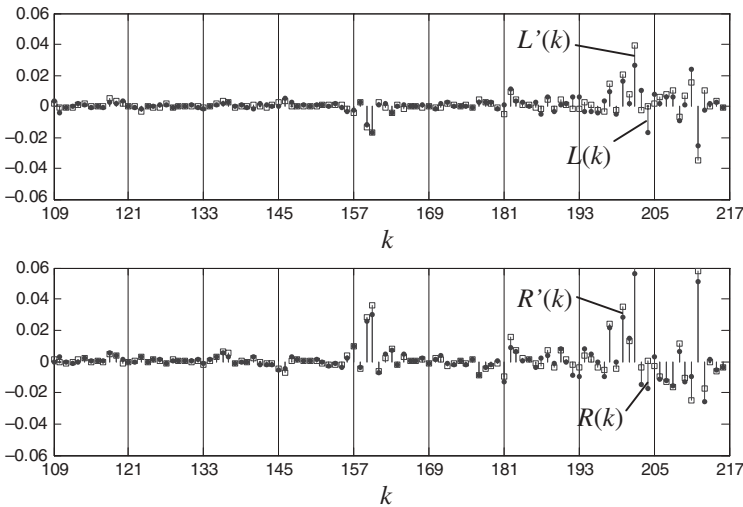


Figure 10.12 Original and reconstructed frequency coefficients for the left and right side channels shown in Figure 10.7.

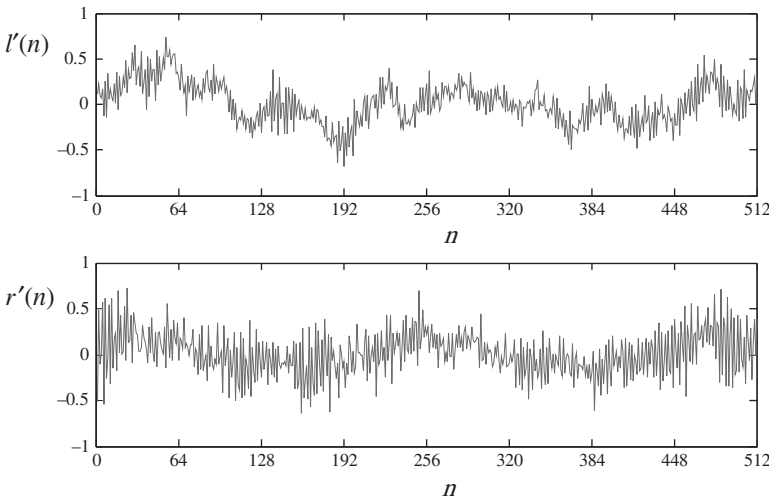


Figure 10.13 Reconstructed audio signals for the left and right side channels shown in Figure 10.7. ■

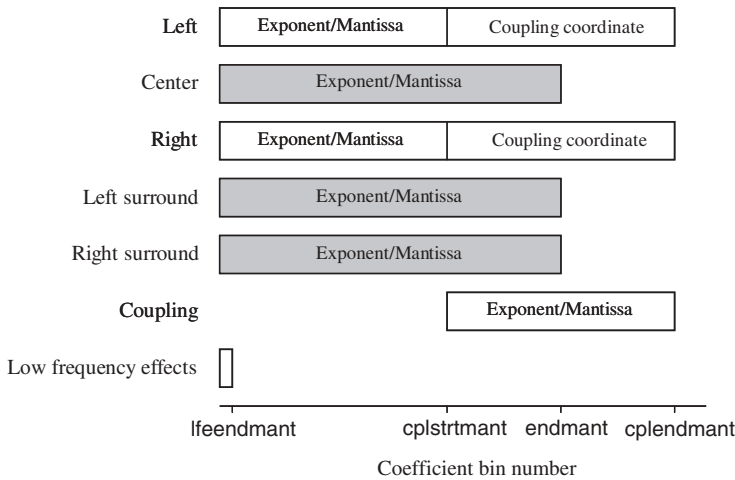


Figure 10.14 Structure of the channels to be coded when coupling is in use.

So, after coupling has been performed, there are a number of different types of channels that are transmitted to the receiver. Consider the example shown in Figure 10.14 where only the left and right side channels are included in the coupling process. The coefficients of the *coupled* channels (the left and right side channel in this example) are coded individually as exponent and mantissa values up to the coefficient bin number indicated by the parameter `cplstrtmant`. Then above this bin number, the coefficients are coded using coupling parameters up to the bin number indicated by `cplendmant`. The coefficients of the *coupling* channel are only required for bin numbers between `cplstrtmant` and `cplendmant` and are coded individually as exponent and mantissa values. The remaining full bandwidth channels that are not included in the coupling process are referred to as *independent* channels, and these are shown as shaded parts in Figure 10.14. The coefficients of the independent channels are coded individually as exponent and mantissa values up to the bin number indicated by `fbwendmant[ch]`. Note that the value of `fbwendmant[ch]` may be different for each independent channel. Finally the coefficients in the low-frequency effects channel are coded individually as exponent and mantissa values up to the bin number indicated by `lfeendmant`. The starting bin numbers for the full bandwidth and LFE channels are indicated by `fbwstrtmant[ch]` and `lfestrtmant`, respectively.

10.1.5. Rematrixing

Rematrixing is an additional channel combination technique that is employed in the 2/0 audio coding mode. When rematrixing is in use, the coder is able to transmit a $L + R$ signal and a $L - R$ signal instead of the original L and R stereo signals. If coupling is also in use, rematrixing is only performed on those frequencies below the coupling frequency.

The use of rematrixing allows a significant reduction in the bit rate when the original left and right side signals are highly correlated. Rematrixing is also important for preserving compatibility with Dolby surround algorithms. Consider the case where the input left and right side signals are transmitted without rematrixing. The received left and right side signals, denoted by L_r and R_r , are

$$\begin{aligned} L_r &= L + N_1 \\ R_r &= R + N_2 \end{aligned} \quad (10.3)$$

where N_1 and N_2 denote the uncorrelated quantization noise associated with the encoding process. The Dolby Pro Logic decoder takes these received signals and constructs a center and surround channel, denoted by C and S , respectively, which are as follows:

$$\begin{aligned} C &= 0.5(L + R + (N_1 + N_2)) \\ S &= 0.5(L - R + (N_1 - N_2)) \end{aligned} \quad (10.4)$$

So, for the decoded center channel, the noise signals add, but the resulting noise remains masked by $L + R$ since this signal is of similar magnitude to the original left and right side signals. However, for the decoded surround channel, because the noise signals are uncorrelated they also add, but in this case the $L - R$ signal is much smaller in magnitude than the original left and right side signals and may not mask the decoded noise signal.

Now consider the case when rematrixing is in use. The received left and right side signals are

$$\begin{aligned} L_r &= 0.5(L + R) + N_1 \\ R_r &= 0.5(L - R) + N_2 \end{aligned} \quad (10.5)$$

Note that, in this case, the encoding process attempts to mask all quantization noise in the $L - R$ signal, and hence N_2 is typically much smaller in magnitude than N_1 . The Dolby Pro Logic decoder takes these received signals and constructs the center and surround channel as follows:

$$\begin{aligned} C &= L_r \\ S &= R_r \end{aligned} \quad (10.6)$$

Hence the quantization noise remains masked for the decoded center and surround signals. The decoded left and right side signals, denoted by L_d and R_d , are

$$\begin{aligned} L_d &= L_r + R_r = L + (N_1 + N_2) \\ R_d &= L_r - R_r = R + (N_1 - N_2) \end{aligned} \quad (10.7)$$

So for both these signals the uncorrelated noise signals add but are dominated by the much larger N_1 . Hence most, if not all, of the decoded noise remains masked by the L and R signals, which are of similar magnitude to the $0.5(L + R)$ signal that was used in the encoding process.

10.1.5.1. Rematrixing Frequency Bands

The rematrixing process is conducted using independent frequency bands. The boundary locations for these bands are defined using the coefficient numbers and depend on the number of coefficients that are included in the coupling process. Tables 10.8 to 10.11 show the four rematrixing band boundaries to be used for different values of **cplbegf**.

Table 10.8 Rematrixing bands, coupling not in use.

Band No.	Start Coeff.	End Coeff.
0	13	24
1	25	36
2	37	60
3	61	252

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.9 Rematrixing bands, **cplbegf** > 2.

Band No.	Start Coeff.	End Coeff.
0	13	24
1	25	36
2	37	60
3	61	$36 + \mathbf{cplbegf} \times 12$

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.10 Rematrixing bands, $2 \geq \mathbf{cplbegf} > 0$.

Band No.	Start Coeff.	End Coeff.
0	13	24
1	25	36
2	37	$36 + \mathbf{cplbegf} \times 12$

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.11 Rematrixing bands, **cplbegf** = 0.

Band No.	Start Coeff.	End Coeff.
0	13	24
1	25	36

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.1.5.2. Encoding Technique

In the 2/0 audio coding mode, rematrixing is always in use. However, for each rematrixing band it is still possible to transmit the original L and R channels rather than rematrixing them into sum and difference channels. The decision to rematrix or not is made using the sum of the squares of the coefficients in the band. This sum is calculated for the L and R channels and for the $L + R$ and $L - R$ combinations. If the minimum of these four sums for a rematrixing band corresponds to the L or R channels, then L and R channels are transmitted for that band. Alternatively, if the minimum sum corresponds to the $L + R$ or the $L - R$ combination, then the rematrixed sum and difference channels $0.5(L + R)$ and $0.5(L - R)$ are transmitted.

The type of channel to be transmitted for each band is indicated by the parameter **rematflg[rband]**. This parameter is treated as an array of single bit elements with one element for each rematrixing band. A value of 1 for a bit in **rematflg[rband]** indicates that the coefficients in the band have been rematrixed into sum and difference channels.

10.1.6. Extract Exponents

After the coupling (and rematrixing if in 2/0 mode) strategies have been determined, the coefficients that are to be transmitted to the decoder are known. These coefficients are transmitted in floating point form as a mantissa and an exponent value.

The exponent information is coded differentially across the transmitted frequency range for the channel. The first exponent of the channel is sent as a 4-bit absolute value with a range of 0–15. Then successive exponent values are transmitted as differential values. These differential values are referred to as *differential exponents* and are limited to one of the following five values: -2 , -1 , 0 , 1 , or 2 . The decoded differential exponents can be used to form an approximation of the power spectral density of the audio signal. This approximation is commonly referred to as the *coded spectral envelope* of the signal.

The differential exponents can be calculated using three different methods that are referred to as *exponent strategies*. The difference between the three methods is simply the number of original exponents corresponding to each differential exponent. The first method is called the D-15 exponent strategy, and in this method

a differential exponent is transmitted for every original exponent value. The D-15 strategy results in the highest resolution spectral envelope and also requires the most number of bits to transmit. For this reason the D-15 strategy is typically used for signals with a spectral envelope that remains relatively stable over time. For this type of signal, the coded spectral envelope is usually transmitted once at the beginning of each audio frame and is reused for each of the six audio blocks in the frame.

EXAMPLE 10.3

Figure 10.15 shows the original exponent values for the first 36 coefficients of Figure 10.6 together with the differentially coded spectral envelope for the signal when the D-15 exponent strategy is in use. Figure 10.16 shows the differential exponents that would be transmitted to the decoder for the spectral envelope shown in Figure 10.15.

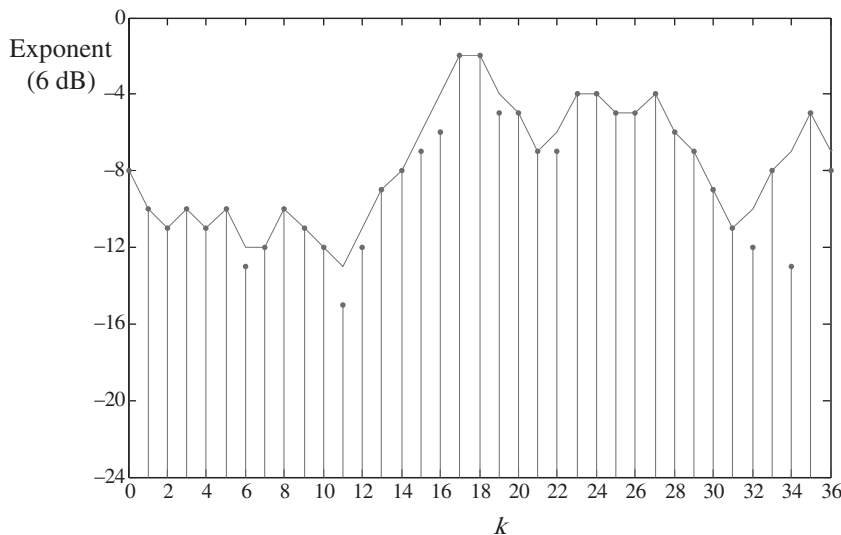


Figure 10.15 Original exponent values and the corresponding differentially coded spectral envelope for the first 36 coefficients shown in Figure 10.6.

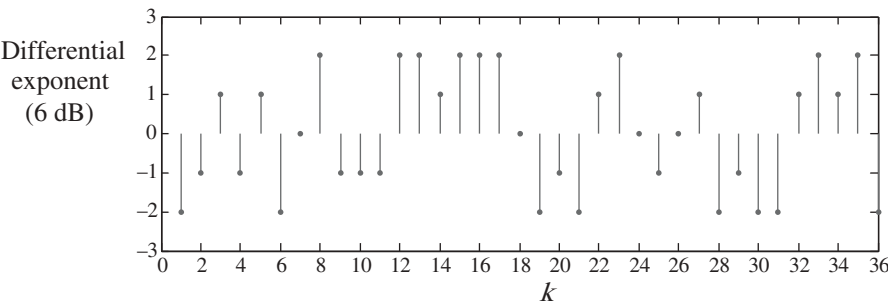


Figure 10.16 Transmitted differential exponents for the spectral envelope shown in Figure 10.15.

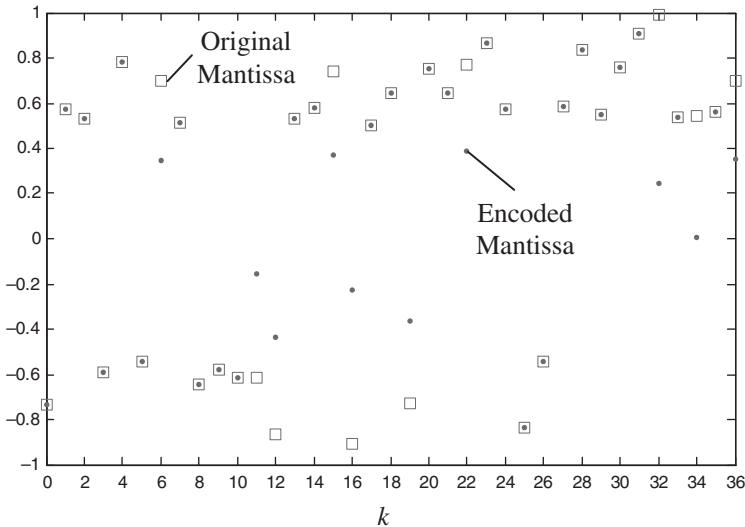


Figure 10.17 Original mantissa values and the mantissa values to be quantized and transmitted for the first 36 coefficients of Figure 10.5.

Figure 10.15 shows that the limited range of possible differential exponent values means that the coded spectral envelope may not always exactly match the original exponent values. However, if the coded spectral envelope is always greater than or equal to the actual exponent values, then the discrepancy simply means that the mantissa value to be quantized and transmitted is smaller than the original mantissa value. To illustrate this point, Figure 10.17 shows the original mantissa values for the first 36 coefficients of Figure 10.5 together with the mantissa values obtained by dividing the same 36 coefficients by the coded spectral envelope shown in Figure 10.15. ■

The second method is called the D-25 exponent strategy, and in this method a differential exponent is transmitted for every two original exponent values. The D-25 strategy results in a medium-resolution spectral envelope and is typically used for signals with a spectral envelope that remains relatively stable for two to three audio blocks. Hence, if the D-25 exponent strategy is in use, the coded spectral envelope is usually transmitted twice for each audio frame and the same spectral envelope is used for three consecutive audio blocks in the frame.

The final method is called the D-45 exponent strategy, and in this method a differential exponent is transmitted for every four original exponent values. The D-45 strategy results in a low-resolution spectral envelope and is typically used for transient signals where the spectral envelope changes for each block but is usually relatively flat in nature. Consequently, a coded spectral envelope is usually transmitted for every audio block if the D-45 strategy is in use.

EXAMPLE 10.4

Figure 10.19 shows the coded spectral envelope produced using the D-45 exponent strategy for the coefficients of the transient signal shown in Figure 10.18.

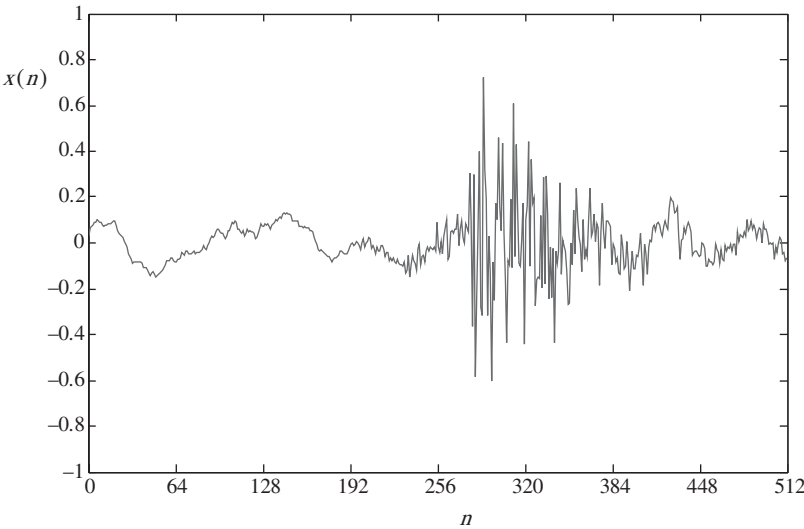


Figure 10.18 Transient audio signal produced by a castanet “click.”

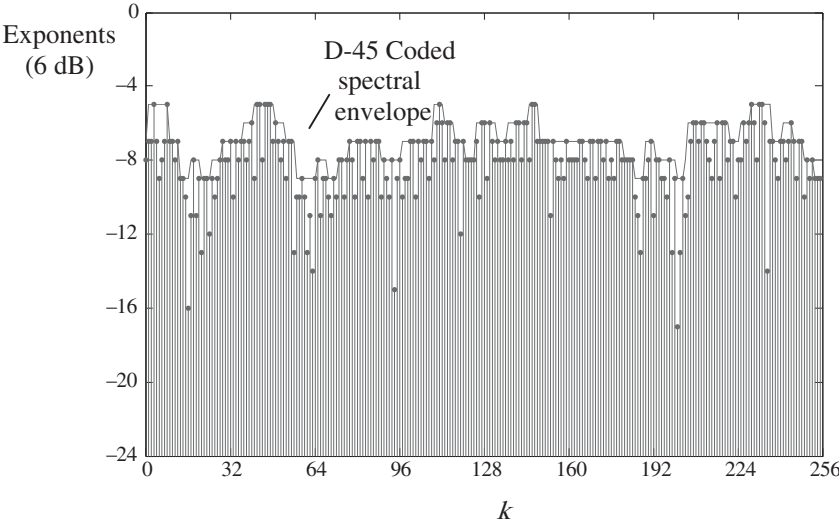


Figure 10.19 Original exponent values and the D-45 coded spectral envelope for the coefficients of the transient signal shown in Figure 10.18. ■

Table 10.12 Exponent strategy code words.

chexpstr[ch], cplexpstr	Exponent strategy
“00”	Reuse prior exponents
“01”	D15
“10”	D25
“11”	D45

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

The exponent strategy in use for a full bandwidth channel is transmitted to the decoder using a 2-bit code word for each channel designated by **chexpstr[ch]**. If the coupling channel is present, the 2-bit code word **cplexpstr** is used. The exponent strategy corresponding to each value of these code words is shown in Table 10.12 for full bandwidth and coupling channels.

If the low-frequency effects channel is enabled, the exponent strategy is transmitted to the decoder using a single bit code word designated by **lfeexpstr**. A value of 1 for **lfeexpstr** indicates that a new spectral envelope coded using the D-15 exponent strategy will be transmitted for the current audio block, otherwise the spectral envelope from the previous block should be reused.

If a full bandwidth channel is not included in the coupling process, it is often not necessary to transmit the entire set of coefficients for the channel. In such cases, the last coefficient to be transmitted is indicated by a *channel bandwidth code*, designated by **chbwcod[ch]**. The channel bandwidth code is transmitted to the decoder as a 6-bit unsigned integer and is calculated from the end mantissa bin number, **endmant[ch]**, using the following formula:

$$\mathbf{chbwcod[ch]} = ((\mathbf{endmant[ch]} - 37)/3) + 12 \quad (10.8)$$

Valid values for **chbwcod[ch]** lie in the range 0–60.

10.1.7. Encode Exponents

The spectral envelope for a channel is transmitted using a 4-bit absolute value for the first exponent of the channel followed by a set of differential exponents. For full bandwidth and low-frequency effects channels, the initial absolute value is limited to the range 0 to 15, and exponent values larger than 15 are truncated. For the coupling channel, the absolute exponent is limited to even values and is divided by a factor of 2 before transmission. The initial absolute exponent value is transmitted to the decoder using a 4-bit unsigned integer designated by **cplabsexp**, **exps[ch][0]**, and **lfeexps[0]** for the coupling, full bandwidth, and low-frequency effects channels, respectively.

Each differential exponent is first mapped to a positive integer, called a *mapped value*, by adding a factor of 2. Then each set of three adjacent mapped values is grouped into a 7-bit *grouped value* using the formula:

$$G = 25M_1 + 5M_2 + M_3 \quad (10.9)$$

where G denotes the 7-bit grouped value, and M_1 , M_2 , and M_3 are three adjacent mapped values. The grouped values are transmitted to the decoder using 7-bit unsigned integers designated by **cplexps[grp]**, **exps[ch][grp]**, and **lfeexps[grp]** for the coupling, full bandwidth, and low frequency effects channels, respectively.

10.1.8. Bit Allocation

The bit-allocation strategy used in the AC-3 algorithm consists of two separate procedures; a standard *core* bit-allocation procedure and an optional *delta* bit-allocation procedure. The core bit-allocation algorithm uses the coded spectral envelope as an approximation to the power spectral density of the transmitted signal. A standard psychoacoustic model is then used to determine a masking threshold for the signal. This threshold can be used to determine the number of bits to be allocated to each mantissa for a given level of perceived distortion at the receiver. The psychoacoustic model used in the core bit-allocation procedure is defined as part of the standard. The approach used is to iteratively decrease the level of noise introduced by the coding process until the required bit rate is reached and then transmit the final setup of the model to the decoder via a number of parameters in the bit stream. The decoder can then reproduce the identical bit-allocation values for each coefficient and correctly decode the transmitted coefficients.

The advantage of this approach is that it is not necessary to transmit the actual bit allocation for each coefficient in the bit stream. The decoder has the coded spectral envelope and the parameters of the psychoacoustic model that were used in the encoding process, so it can repeat the calculations and arrive at identical bit-allocation values. The disadvantage of this approach is that the psychoacoustic model is fixed and cannot be improved or changed for different input signals. To overcome this problem, the syntax allows for delta bit-allocation parameters to be transmitted so that the bit-allocation values obtained by the standard psychoacoustic model can be adjusted to match the output of an alternative model. These adjustments should only be minor because the standard psychoacoustic model usually provides a good approximation to the required acoustic model.

10.1.8.1. Initialization

The first step in the core bit-allocation process is to initialize the **strtmant** and **endmant** parameters for each of the channels. These parameters are determined using **chincpl[ch]** for each full bandwidth channel, **cplbegf** and **cplendf** for the coupled and coupling channels, and **chbwcod[ch]** for the independent channels. The following MATLAB code defines the algorithm used to initialize these parameters. The value of the parameter **nfchans** is equal to the number of full bandwidth channels and is determined from **acmod** as shown in Table 10.2.

```

for ch = 0:nfchans-1
    fbwstrtmant(ch+1) = 0;
    if chincpl(ch+1)
        fbwendmant(ch+1) = 37 + (12*cplbegf);
    else
        fbwendmant(ch+1) = 37 + (3*(chbwcod(ch+1) + 12));
    end
end

cplstrtmant = 37 + (12*cplbegf);
cplendmant = 37 + (12*(cplendf + 3));

lfestrtmant = 0;
lfeendmant = 7;

```

Then, in the following procedures, the parameters strtmant and endmant are equal to fbwstrtmant[ch] and fbwendmant[ch] for the full bandwidth channels, cplstrtmant and cplendmant for the coupling channel, and lfestrtmant and lfeendmant for the LFE channel.

10.1.8.2. Exponent to PSD Conversion

The next step in the core bit-allocation procedure is to convert the coded spectral envelope to a power spectral density function. The conversion process consists of simply shifting the range of coded spectral envelope values from $[-24, 0]$ to $[0, 3072]$. The algorithm used is defined by the following MATLAB code:

```

for bin = strtmant:endmant
    psd(bin+1) = 3072 - exponent(bin+1)*128;
end

```

10.1.8.3. PSD Integration

The next step in the process is to convert the domain of the PSD values from a fine-resolution linear frequency scale into a scale that approximates the Bark scale. To perform this conversion, the frequency scale is subdivided into 50 nonuniform frequency *bands*. The power spectral density value for each band, designated by bndpsd[band], is found by calculating the integral of the PSD values in the band. The width of the bands is nonuniform and is derived from the critical bandwidths at the corresponding frequency. The starting bin number for each band is defined using the array bndtab[band], and the width of each band is defined using the array bndsz[band]. The values contained in these two arrays are shown in Table 10.13.

The conversion from a linear frequency scale into bands corresponding to the Bark scale requires the integration of the linear values corresponding to the logarithmic PSD values. So a direct computation of bndpsd[band] would require the

Table 10.13 Bark scale banding structure.

band	bndtab[band]	bndsz[band]	band	bndtab[band]	bndsz[band]
0	0	1	25	25	1
1	1	1	26	26	1
2	2	1	27	27	1
3	3	1	28	28	3
4	4	1	29	31	3
5	5	1	30	34	3
6	6	1	31	37	3
7	7	1	32	40	3
8	8	1	33	43	3
9	9	1	34	46	3
10	10	1	35	49	6
11	11	1	36	55	6
12	12	1	37	61	6
13	13	1	38	67	6
14	14	1	39	73	6
15	15	1	40	79	6
16	16	1	41	85	12
17	17	1	42	97	12
18	18	1	43	109	12
19	19	1	44	121	12
20	20	1	45	133	24
21	21	1	46	157	24
22	22	1	47	181	24
23	23	1	48	205	24
24	24	1	49	229	24

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

conversion of the logarithmic PSD values into linear values, then a summation, and finally a conversion back to a logarithmic scale for the banded PSD values. However, because the logarithmic PSD values are available, this process of *log addition* can be performed more efficiently if the following relationship is used:

$$\log_2(a + b) = \max(\log_2(a), \log_2(b)) + \log_2\left(1 + 2^{-|\log_2(a) - \log_2(b)|}\right) \quad (10.10)$$

Equation (10.10) shows that if a and b are dissimilar and hence $|\log(a) - \log(b)|$ is large, then the second term on the right-hand side becomes insignificant. Consequently, $\log_2(a + b)$ is dominated by the larger values of a and b and can be approximated by $\max(\log_2(a), \log_2(b))$. Alternatively, if a and b are similar then the second term on the right-hand side is significant and provides the required adjustment

to $\max(\log_2(a), \log_2(b))$. The maximum adjustment is required when a and b are equal, in which case the second term becomes $\log_2(1 + 1) = 1$.

In the AC-3 encoder, the log addition is performed using a maximum operation for the first term on the right-hand side of Equation (10.10) and a subtraction, absolute value, and table lookup for the second term. The output of the table lookup operation is the integer part of $64 \log_2(1 + 2^{-d/32})$, if d is the index to the table. So the algorithm used to calculate the banded PSD values is defined by the following MATLAB code. The values in the lookup table, `latab[]`, used in the log-addition process are shown in Table 10.14, and the table used to convert bin numbers to band numbers, `masktab[]`, is shown in Table 10.15.

Table 10.14 Log addition lookup table values, `latab[i+j]`.

	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$	$j = 8$	$j = 9$
$i = 0$	64	63	62	61	60	59	58	57	56	55
$i = 10$	54	53	52	52	51	50	49	48	47	47
$i = 20$	46	45	44	44	43	42	41	41	40	39
$i = 30$	38	38	37	36	36	35	35	34	33	33
$i = 40$	32	32	31	30	30	29	29	28	28	27
$i = 50$	27	26	26	25	25	24	24	23	23	22
$i = 60$	22	21	21	21	20	20	19	19	19	18
$i = 70$	18	18	17	17	17	16	16	16	15	15
$i = 80$	15	14	14	14	13	13	13	13	12	12
$i = 90$	12	12	11	11	11	11	10	10	10	10
$i = 100$	10	9	9	9	9	9	8	8	8	8
$i = 110$	8	8	7	7	7	7	7	7	6	6
$i = 120$	6	6	6	6	6	6	5	5	5	5
$i = 130$	5	5	5	5	4	4	4	4	4	4
$i = 140$	4	4	4	4	4	3	3	3	3	3
$i = 150$	3	3	3	3	3	3	3	3	3	2
$i = 160$	2	2	2	2	2	2	2	2	2	2
$i = 170$	2	2	2	2	2	2	2	2	1	1
$i = 180$	1	1	1	1	1	1	1	1	1	1
$i = 190$	1	1	1	1	1	1	1	1	1	1
$i = 200$	1	1	1	1	1	1	1	1	1	1
$i = 210$	0	0	0	0	0	0	0	0	0	0
$i = 220$	0	0	0	0	0	0	0	0	0	0
$i = 230$	0	0	0	0	0	0	0	0	0	0
$i = 240$	0	0	0	0	0	0	0	0	0	0
$i = 250$	0	0	0	0	0	0	0	0	0	0

Table 10.15 Bin number to band number conversion table, masktab[i+j].

	<i>j</i> = 0	<i>j</i> = 1	<i>j</i> = 2	<i>j</i> = 3	<i>j</i> = 4	<i>j</i> = 5	<i>j</i> = 6	<i>j</i> = 7	<i>j</i> = 8	<i>j</i> = 9
<i>i</i> = 0	0	1	2	3	4	5	6	7	8	9
<i>i</i> = 10	10	11	12	13	14	15	16	17	18	19
<i>i</i> = 20	20	21	22	23	24	25	26	27	28	28
<i>i</i> = 30	28	29	29	29	30	30	30	31	31	31
<i>i</i> = 40	32	32	32	33	33	33	34	34	34	35
<i>i</i> = 50	35	35	35	35	35	36	36	36	36	36
<i>i</i> = 60	36	37	37	37	37	37	37	38	38	38
<i>i</i> = 70	38	38	38	39	39	39	39	39	39	40
<i>i</i> = 80	40	40	40	40	40	41	41	41	41	41
<i>i</i> = 90	41	41	41	41	41	41	41	42	42	42
<i>i</i> = 100	42	42	42	42	42	42	42	42	42	43
<i>i</i> = 110	43	43	43	43	43	43	43	43	43	43
<i>i</i> = 120	43	44	44	44	44	44	44	44	44	44
<i>i</i> = 130	44	44	44	45	45	45	45	45	45	45
<i>i</i> = 140	45	45	45	45	45	45	45	45	45	45
<i>i</i> = 150	45	45	45	45	45	45	45	46	46	46
<i>i</i> = 160	46	46	46	46	46	46	46	46	46	46
<i>i</i> = 170	46	46	46	46	46	46	46	46	46	46
<i>i</i> = 180	46	47	47	47	47	47	47	47	47	47
<i>i</i> = 190	47	47	47	47	47	47	47	47	47	47
<i>i</i> = 200	47	47	47	47	47	48	48	48	48	48
<i>i</i> = 210	48	48	48	48	48	48	48	48	48	48
<i>i</i> = 220	48	48	48	48	48	48	48	48	48	49
<i>i</i> = 230	49	49	49	49	49	49	49	49	49	49
<i>i</i> = 240	49	49	49	49	49	49	49	49	49	49
<i>i</i> = 250	49	49	49	0	0	0				

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

```

bin = strtmant;
band = masktab(strtmant11);
lastbin = bin;
while (lastbin < endmant)

    % calculate the last coefficient bin number in the current critical
    % band
    lastbin = min(bndtab(band+1) + bndsz(band+1), endmant);

    % find the PSD values for the current band using the process of log
    % addition
    bndpsd(band+1) = psd(bin+1);
    bin = bin+1;

```

```

for i = bin:lastbin-1
    bndpsd(band+1) = logadd(bndpsd(band+1),psd(bin+1));
    bin = bin+1;
end
band = band+1;
end

function x = logadd(a,b)

c = a - b;

% find the address into the look up table latab using the difference in the
% PSD values
d = min(floor(abs(c)/2),255);

if (c >= 0)
    x = a + latab(d+1);
else
    x = b + latab(d+1);
end

```

10.1.8.4. The Spreading Function

The banded PSD values are then used to estimate the masking threshold curve by convolving them with a *spreading function*. The spreading function is approximated by a fast-decaying upward masking curve and a slow-decaying upward masking curve as shown in Figure 10.20. Downward masking is ignored in this algorithm to reduce computational complexity at the expense of calculating a more conservative masking threshold.

A simplified convolution is then performed using the two masking curves and the banded PSD values. The calculations for the convolution operation are carried out using the logarithmic amplitude values available in the **bndpsd[band]** array. This simplifies the process since, in the log domain, multiplication can be

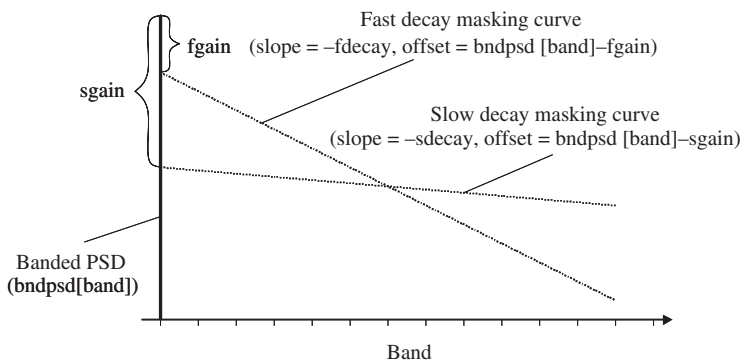


Figure 10.20 Linear masking curves used to approximate the upward masking spreading function.

performed by addition, and log additions can be approximated by a maximum value operation.

To explain this simplification further, consider the case of performing a linear convolution of a signal x_n and a filter with exponentially decaying coefficients $[1, 1/2, 1/4, 1/8]$.

In the linear domain, the output of this convolution is

$$y_n = \frac{x_{n-3}}{8} + \frac{x_{n-2}}{4} + \frac{x_{n-1}}{2} + x_n \quad (10.11)$$

However, if the convolution is performed in the log domain and log addition is replaced with the maximum operator, the output of the convolution is

$$\log_2(y_n) = \max(\log_2(x_{n-3}) - 3, \log_2(x_{n-2}) - 2, \log_2(x_{n-1}) - 1, \log_2(x_n)) \quad (10.12)$$

For the special case of a filter with exponentially decaying coefficients, the process can be simplified further by performing the maximum operation in a recursive manner. The output of the convolution can now be written as a function of the previous output and is

$$\log_2(y_n) = \max(\log_2(y_{n-1}) - 1, \log_2(x_n)) \quad (10.13)$$

To see why this recursive simplification is possible, consider the previous convolution output given by

$$\log_2(y_{n-1}) = \max(\log_2(x_{n-3}) - 2, \log_2(x_{n-2}) - 1, \log_2(x_{n-1})) \quad (10.14)$$

Then, if both sides of this equation are decayed, the result is

$$\begin{aligned} \log_2(y_{n-1}) - 1 &= \max(\log_2(x_{n-3}) - 2, \log_2(x_{n-2}) - 1, \log_2(x_{n-1})) - 1 \\ &= \max(\log_2(x_{n-3}) - 3, \log_2(x_{n-2}) - 2, \log_2(x_{n-1}) - 1) \end{aligned} \quad (10.15)$$

Hence, the output of the convolution process in the log domain can be obtained by simply finding the maximum of the current input and a decayed version of the previous output.

An example illustrating this simplification process is shown in Figure 10.21. Figure 10.21 shows that, when $n = 3$, the maximum of the decayed values of the previous inputs ($\log_2(x_0) - 3$) is equal to the decayed previous output value ($\log_2(y_2) - 1$). So the output of the convolution for $n = 3$ is given by the maximum of $\log_2(y_2) - 1$ and $\log_2(x_3)$.

This simplified convolution process is conducted for the two linear masking curves, and the final output is referred to as the *excitation function* for the current set

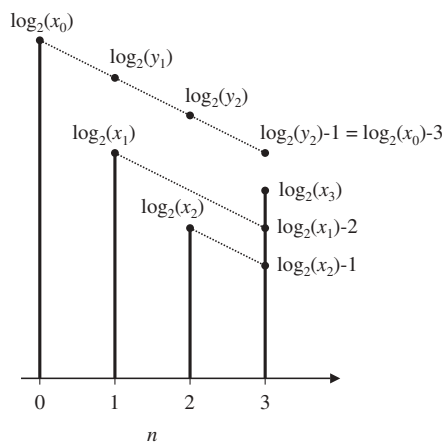


Figure 10.21 Illustration of the simplification process for performing convolution in the log domain with exponentially decaying filter coefficients.

of banded PSD values. The convolution process to determine the excitation function is illustrated in Figure 10.22. The output of the convolution of the fast-decaying masking curve with the banded PSD values is stored in the fastleak parameter as shown in Figure 10.22(a). The output of the convolution of the slow-decaying masking curve

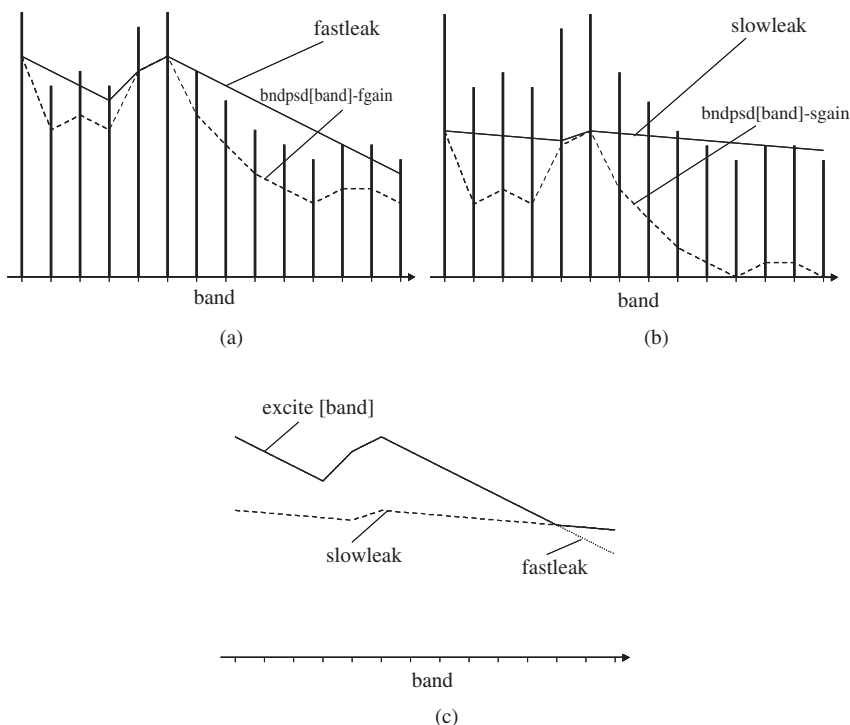


Figure 10.22 An example illustrating the convolution process used to form the excitation function.

with the banded PSD values is stored in the `slowleak` parameter as shown in Figure 10.22(b). The final excitation function, designated by `excite[band]`, is calculated by taking the maximum of the `fastleak` and `slowleak` values for each band as shown in Figure 10.22(c).

10.1.8.5. Compensation for Decoder Selectivity

A problem exists for the lower frequency coefficients of the decoder filterbank. At these low frequencies there is a significant amount of overlap between the subbands of the decoder filterbank. So if quantization distortion is introduced into one band at the encoder, then a significant amount of this distortion appears in neighboring frequencies at the output of the decoder filter bank. The problem occurs predominantly at low frequencies where the slope of the masking curve can exceed the roll-off of the filter frequency response.

In the AC-3 encoder a compensation for this poor decoder frequency selectivity is incorporated into the calculation of the excitation function. The compensation is only applied to frequencies between 0 and approximately 700 Hz, and this range is further subdivided into two smaller frequency ranges. For frequencies below about 200 Hz, no upward masking is assumed when calculating the excitation function. For frequencies between 200 and 700 Hz, upward masking is only enabled for frequencies above the first significant spectral component.

In addition to these limitations on upward masking, the compensating factor, `lowcomp`, is included in the calculation of the excitation function for frequencies between 0 and about 2.3 kHz. The value for `lowcomp` is calculated using the slope between consecutive banded PSD values using the algorithm defined by the following MATLAB code.

```
% for frequencies below about 700 Hz
if band < 7
    % if two consecutive PSD values have a difference of 256 (+12 dB)
    % the value of lowcomp is set to 384 (an 18 dB adjustment)
    if bandpsd0 + 256 == bandpsd1
        lowcomp = 384;
    % if two consecutive PSD values have a difference of -128 or -256 (-6
    % or -12 dB) the value for lowcomp is taken as the larger of: the
    % current value of lowcomp minus 64 and zero
    elseif bandpsd0 > bandpsd1
        lowcomp = max(0, lowcomp - 64);
    end

    % if two consecutive PSD values have a difference of 128 or 0 (+6 or 0
    % dB) the value for lowcomp remains unchanged
% for frequencies between about 700 Hz and 1.8 kHz the algorithm is the
% same except that the value of lowcomp is set to 320 (a 12 dB adjustment)
elseif band < 20
```

```

if bandpsd0 + 256 == bandpsd1
    lowcomp = 320;
elseif bandpsd0 > bandpsd1
    lowcomp = max(0,lowcomp - 64);
end

% for frequencies between about 1.8 kHz and 2.3 kHz, the value for lowcomp
% is taken as the larger of: the current value of lowcomp minus 128, and
% zero

else
    lowcomp = max(0,lowcomp - 128);
end

```

For frequencies below about 700 Hz, if any two consecutive PSD values have a difference of 256 (i.e. a positive slope of 12 dB), the value of lowcomp is set to 384 (i.e., an 18-dB adjustment). If two consecutive PSD values have a difference of 128 or 0 (i.e., a positive slope of 6 or 0 dB), the value for lowcomp remains unchanged. If two consecutive PSD values have a difference of -128 or -256 (i.e., a slope of -6 or -12 dB), the value for lowcomp is taken as the larger of the current value of lowcomp minus 64 and zero.

For frequencies between about 700 Hz and 1.8 kHz, the algorithm is the same except that the value of lowcomp is set to 320 for a slope of 12 dB and, for frequencies between about 1.8 and 2.3 kHz, the value for lowcomp is taken as the larger of the current value of lowcomp minus 128 and zero.

So the full algorithm to compute the excitation function, including low-frequency selectivity compensation, is defined by the following MATLAB code:

```

bndstrt = masktab(strtmant+1);
bndend = masktab(endmant) + 1;
% for fbw and lfe channels
if (bndstrt == 0)
    % for frequencies below about 200 Hz, no upward masking is assumed when
    % calculating the excitation function.
    lowcomp = calc_lowcomp(lowcomp,bndpsd(1),bndpsd(2),0);
    excite(1) = bndpsd(1) - fgain - lowcomp;
    lowcomp = calc_lowcomp(lowcomp,bndpsd(2),bndpsd(3),1);
    excite(2) = bndpsd(2) - fgain - lowcomp;
    begin = 7;

    % for frequencies between 200 and 700 Hz, upward masking is only
    % enabled for frequencies above the first significant spectral
    % component.

    for band = 2:6
        % skip for the last band of the lfe channel
        if ((bndend ~= 7) || (band ~= 6))

```

```

        lowcomp = calc_lowcomp(lowcomp,bndpsd(band+1),bndpsd(band+2),band);
    end
    % calculate the starting values for the convolution
    fastleak = bndpsd(band+1) - fgain;
    slowleak = bndpsd(band+1) - sgain;
    % but don't perform masking until a significant spectral component
    % is found
    excite(band+1) = bndpsd(band+1) - fgain - lowcomp;
    % skip for the last band of the lfe channel
    if ((bndend ~= 7) || (band ~= 6))
        % if a significant spectral component is found begin upward
        % masking
        if (bndpsd(band+1) <= bndpsd(band+2))
            begin = band + 1;
            break;
        end
    end
end
for band = begin:min(bndend,22)-1
    % skip for the last band of the lfe channel
    if ((bndend ~= 7) || (band ~= 6))
        lowcomp = calc_lowcomp(lowcomp,bndpsd(band+1),bndpsd(band+2),band);
    end
    % perform the convolution in the log domain for the slow decaying
    % masking curve
    fastleak = fastleak - fdecay;
    fastleak = max(fastleak, bndpsd(band+1) - fgain);
    % perform the convolution in the log domain for the fast decaying
    % masking curve
    slowleak = slowleak - sdecay;
    slowleak = max(slowleak, bndpsd(band+1) - sgain);
    % the excitation function is taken as the maximum of the fast
    % decaying and slow decaying masking curves
    excite(band+1) = max(fastleak - lowcomp, slowleak);
end
begin = 22;
% for the coupling channel
else
    begin = bndstrt ;
end
for band = begin:bndend-1
    fastleak = fastleak - fdecay;

```

```
fastleak = max(fastleak, bndpsd(band+1) - fgain);
slowleak = slowleak - sdecay;
slowleak = max(slowleak, bndpsd(band+1) - sgain);
excite(band+1) = max(fastleak, slowleak);
end
```

where strtmant is equal to fbwstrtmant[ch], cplstrtmant, or lfestrtmant and endmant is equal to fbwendmant[ch], cplendmant, or lfeendmant for the full bandwidth, coupling, and LFE channels, respectively.

The values for the parameters sdecay, fdecay, and sgain are constant for all channels and are transmitted to the decoder using the code words **sdcycod**, **fdccod**, and **sgaincod**. These code words are treated as addresses into the corresponding lookup tables shown in Table 10.16.

The value for the parameter fgain may vary with the type of channel and is transmitted to the decoder using the code words **fgaincod[ch]**, **cplfgaincod**, and **lfefgaincod** for the full bandwidth, coupling, and LFE channels, respectively. These code words are treated as addresses into the lookup table shown in Table 10.17.

For the coupling channel, initial values for the fastleak and slowleak parameter other than zero may be used. These nonzero values are transmitted to the decoder using the 3-bit unsigned integers, **cplfleak** and **cplisleak**. The values for these parameters are calculated from the initial values of fastleak and slowleak using the following equations:

$$\text{cplfleak} = (\text{fastleak} - 768) \gg 8 \tag{10.16}$$

$$\text{cplisleak} = (\text{slowleak} - 768) \gg 8 \tag{10.17}$$

10.1.8.6. Masking Curve Computation

Once the excitation function has been determined, the next step in the bit-allocation process is to compute the masking threshold curve. This curve is calculated using the absolute threshold of hearing, the excitation function, and the parameter dbknee. This last parameter is used to adjust the masking curve according to the absolute level of the masking signal. The results of experimental

Table 10.16 Bit allocation tables slowdec[], fastdec[], and slowgain[].

sdcycod	slowdec[]	fdccod	fastdec[]	sgaincod	slowgain[]
“00”	0x0f	“00”	0x3f	“00”	0x540
“01”	0x11	“01”	0x53	“01”	0x4d8
“10”	0x13	“10”	0x67	“10”	0x478
“11”	0x15	“11”	0x7b	“11”	0x410

Table 10.17 Bit allocation table fastgain[].

fgaincod	fastgain[]
“000”	0x080
“001”	0x100
“010”	0x180
“011”	0x200
“100”	0x280
“101”	0x300
“110”	0x380
“111”	0x400

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

listening tests have shown that the masking threshold increases by about 5 dB for each 20 dB reduction in the absolute level of the masking tone. The parameter dbknee is used to indicate the level below which the masking threshold is adjusted upward. If the masking signal is less than dbknee, the masking threshold is adjusted upward by an amount proportional to the difference between the masking signal and dbknee. If the masking signal is greater than dbknee, the masking curve is not adjusted. The value of dbknee can be set to zero so that no adjustment for masking signal level is included in the calculation. The algorithm used to determine the masking threshold curve is defined by the following MATLAB code:

```

bndstrt = masktab(strtmant+1)
bndend = masktab(endmant)
for band = bndstrt:bndend
    % if the masking signal is less than dbknee, the masking threshold is
    % adjusted upwards by an amount proportional to the difference between
    % the masking signal and dbknee.
    if bndpsd(band+1) < dbknee
        excite(band+1) = excite(band+1) + (dbknee - bndpsd(band+1))/4;
    end
    % if the masking signal is greater than dbknee, the masking curve is
    % not adjusted.
    mask(band+1) = max(excite(band+1), hth(fscod+1,band+1))
end

```

The array hth[fscod][band] contains values for the absolute threshold for each band at each input sampling rate and is shown in Table 10.18.

The value for the parameters dbknee is constant for all channels and is transmitted to the decoder using the code word **dbpbcod**. This code word is treated as an address in the lookup table shown in Table 10.19.

Table 10.18 Absolute threshold table, hth[fscod][band].

fs (kHz)/fscod				fs (kHz)/fscod			
Band	48/0	44.1/1	32/2	Band	48/0	44.1/1	32/2
0	1232	1264	1408	25	832	848	896
1	1232	1264	1408	26	816	832	896
2	1088	1120	1200	27	800	832	880
3	1024	1040	1104	28	784	800	864
4	992	992	1056	29	768	784	848
5	960	976	1008	30	752	768	832
6	944	960	992	31	752	752	816
7	944	944	976	32	752	752	800
8	928	944	960	33	752	752	784
9	928	928	944	34	768	752	768
10	928	928	944	35	784	768	752
11	928	928	944	36	832	800	752
12	928	928	928	37	912	848	752
13	912	928	928	38	992	912	768
14	912	912	928	39	1056	992	784
15	912	912	928	40	1120	1056	816
16	896	912	928	41	1168	1104	848
17	896	896	928	42	1184	1184	960
18	880	896	928	43	1120	1168	1040
19	880	896	928	44	1088	1120	1136
20	864	880	912	45	1088	1088	1184
21	864	880	912	46	1312	1152	1120
22	848	864	912	47	2048	1584	1088
23	848	864	912	48	2112	2112	1104
24	832	848	896	49	2112	2112	1248

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.19 Bit-allocation table dbpbtab[.].

dbpbcod	dbpbtab[]
“00”	0x000
“01”	0x700
“10”	0x900
“11”	0xb00

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.1.8.7. Delta Bit Allocation

The AC-3 coder provides a means of adjusting the masking curve produced by applying the default masking model. This adjustment might be necessary for special types of signals or if a more sophisticated masking model is to be used. The adjustment to the masking curve is achieved by transmitting a set of *delta bit-allocation* parameters as side information. These parameters describe the necessary adjustments to transform the default masking curve into the alternate curve.

The existence of delta bit-allocation parameters in the bit stream is indicated by the single bit code word **deltbaie**. A value of 1 for **deltbaie** indicates that some delta bit-allocation information follows in the bit stream. A value of 0 for **deltbaie** in blocks 2 to 6 indicates that the delta bit-allocation information from the previous block is to be reused. However, if **deltbaie** is 0 in the first block of the audio frame, this indicates that no delta bit-allocation information is to be used in the frame.

The delta bit-allocation strategy to be used in the current block is indicated by the 2-bit code word **deltbae[ch]** for the full bandwidth channels or **cpdeltbae** for the coupling channel as shown in Table 10.20.

The delta bit-allocation parameters describe a set of variable length and constant amplitude *segments* where the amplitude of each segment is the difference between the default masking curve and the alternate curve. The properties of these segments are transmitted as follows.

The number of segments is indicated by a 3-bit unsigned integer **deltnseg[ch]**. The actual number of segments ranges from 1 to 8 and is calculated by adding 1 to the transmitted 3-bit value. For the first segment transmitted (seg = 0), the first band to be modified by that segment is indicated by a 5-bit unsigned integer **deltfst[ch][seg]**. For subsequent segments, this parameter indicates the offset from the last band of the previous segment to the first band of the current segment. The number of bands spanned by each segment is indicated by the 4-bit unsigned integer **deltlen[ch][seg]**. The required adjustment to the default masking curve for the current segment is indicated by the 3-bit code word **deltba[ch][seg]** as shown in Table 10.21.

Table 10.20 Delta bit-allocation strategy.

cpdeltbae/deltbae	Strategy
“00”	Reuse previous state
“01”	New info follows
“10”	Perform no delta allocation
“11”	Reserved

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.21 Delta bit allocation code words and adjustments.

cpldeltba/deltba	Adjustment (dB)
“000”	−24
“001”	−18
“010”	−12
“011”	−6
“100”	6
“101”	12
“110”	18
“111”	24

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

The corresponding information, is transmitted for the coupling channel using the parameters **cpldeltalseg**, **cpldeltoffst[seg]**, **cpldeltlen[seg]**, and **cpldeltba[seg]**.

So if the default masking curve is to be modified using the delta bit-allocation parameters, the following pseudocode defines the algorithm used to make these adjustments for a single channel. The parameters **deltbae**, **deltalseg**, **deltoffst[seg]**, **deltlen[seg]**, and **deltba[seg]** can be replaced by the parameters associated with the channel being coded. The following MATLAB code defines the algorithm used to perform the delta bit-allocation procedure.

```

if ((deltbae == 0) || (deltbae == 1))
    band = 0 ;
    % the number of segments is indicated by deltalseg.
    for seg = 0:deltalseg
        % deltoffst indicates the offset from the last band of the previous
        % segment to the first band of the current segment.
        band = band + deltoffst(seg+1);
        % the required adjustment to the default masking curve for the
        % current segment is indicated by deltba
        if (deltba(seg+1) >= 4)
            delta = (deltba(seg+1) - 3)*128;
        else
            delta = (deltba(seg+1) - 4)*128;
        end
        % The number of bands spanned by each segment is indicated by
        % deltlen
        for k = 0:deltlen(seg+1)-1
            mask(band+1) = mask(band+1) + delta ;

```

```

        band = band + 1;
    end
end
end

```

10.1.8.8. Compute Bit Allocation

In the final step of the bit-allocation process, the signal-to-mask ratio is obtained by subtracting the masking threshold values from the fine resolution PSD values. This signal-to-mask ratio is converted to a 6-bit number and used as an index in a lookup table to obtain the number of bits allocated to each coefficient mantissa. The total number of bits allocated to coding the mantissa values is constrained to be less than or equal to a fixed number of bits available for the current audio frame. The encoder controls the number of bits allocated by adjusting a constant offset, **snroffset**, and a lower bound, **snrfloor**, for the masking threshold values in the array **mask[band]**. The adjustments are made to **snroffset** in an iterative manner until the desired number of bits is produced. The value for **snroffset** at each iteration is calculated using a global coarse offset value, **csnroffset**, and a fine offset value for each full bandwidth channel, **fsnroffset[ch]**, the coupling channel, **cplfsnroffset**, and the low frequency channel, **lfefsnroffset**. The value for **snroffset** for each channel is then calculated from these parameters using Equation (10.18) with the term **fsnroffset** replaced by the fine offset parameter associated with the channel being coded.

$$\text{snroffset} = ((\text{csnroffset} - 15) \times 16 + \text{fsnroffset}) \times 4 \quad (10.18)$$

Once the required number of bits has been allocated to each channel, the coarse offset parameter is transmitted to the decoder as a 6-bit unsigned integer and the fine offset parameters for each channel are transmitted using 4-bit unsigned integers. The following MATLAB code defines the algorithm used to calculate the bit-allocation pointer, designated by **bap[bin]**, for each mantissa value.

```

bin = strtmant;
band = masktab(strtmant+1);
lastbin = bin;

while (endmant > lastbin)
    lastbin = min(bndtab(band+1) + bndsz(band+1),endmant);

    % the encoder controls the number of bits allocated by adjusting a
    % constant offset, snroffset, and a lower bound, snrfloor, for the
    % masking threshold values.

    mask(band+1) = mask(band+1) - snroffset;
    mask(band+1) = mask(band+1) - snrfloor;

```

```

if (mask(band+1) < 0)
    mask(band+1) = 0;
end

mask(band+1) = bitand(mask(band+1),8160);
mask(band+1) = mask(band+1) + snrfloor;

for k = bin:lastbin-1

    % the signal-to-mask ratio is obtained by subtracting the masking
    % threshold values from the fine resolution PSD values.

    address = floor((psd(bin+1) - mask(band+1))/32);

    % This signal to mask ratio is converted to a 6-bit number and used
    % as an index in to a lookup table to obtain the number of bits
    % allocated to each coefficient mantissa.

    address = min(63, max(0, address)) ;
    bap(bin+1) = baptab(address+1);

    bin = bin + 1;
end

band = band+1;
end

```

The lookup table, baptab[smr], to translate the signal-to-mask ratio values into bit-allocation pointers is shown in Table 10.22.

The value for the parameter snrfloor is constant for all channels and is transmitted to the decoder using the code word **floorcod**. This code word is treated as an address in the lookup table shown in Table 10.23.

10.1.9. Quantize Mantissas

Now that the bits to be used for each coefficient number have been allocated, the corresponding mantissa can be quantized with the appropriate level of precision. This quantization is performed using a symmetric uniform quantizer for bit-allocation pointer (bap) values less than 5 and an asymmetric quantizer for bap values greater than 5.

Table 10.24 shows the number of quantizer levels for each bap value and also the number of bits required to code the mantissa. For bap values of 1 and 2, three quantized values are grouped together and coded as 5- and 7-bit code words, respectively, and for the bap value of 4, two quantized values are grouped together and coded as a 7-bit code word.

Before quantization is performed, each mantissa is *normalized* by shifting its binary representation to the left by the number of positions indicated in the corresponding coded exponent value. Then either symmetric or asymmetric quantization is performed on the normalized mantissa values as given in the following section.

Table 10.22 Bit-allocation pointer table.

smr	baptab[smr]	smr	baptab[smr]
0	0	32	10
1	1	33	10
2	1	34	10
3	1	35	11
4	1	36	11
5	1	37	11
6	2	38	11
7	2	39	12
8	3	40	12
9	3	41	12
10	3	42	12
11	4	43	13
12	4	44	13
13	5	45	13
14	5	46	13
15	6	47	14
16	6	48	14
17	6	49	14
18	6	50	14
19	7	51	14
20	7	52	14
21	7	53	14
22	7	54	14
23	8	55	15
24	8	56	15
25	8	57	15
26	8	58	15
27	9	59	15
28	9	60	15
29	9	61	15
30	9	62	15
31	10	63	15

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.1.9.1. Symmetric Quantization ($1 \leq \text{bap} \leq 5$)

For bap values of 1–5, the quantized mantissa value is represented by the code word mantissa_code assigned using a table lookup. The decimal value of the code word assigned to each quantized mantissa value is shown in Tables 10.25–10.29.

Table 10.23 Bit-allocation table floortab[].

floorcod	floortab[]
“000”	0x2f0
“001”	0x2b0
“010”	0x270
“011”	0x230
“100”	0x1f0
“101”	0x170
“110”	0x0f0
“111”	0xf800

© Advanced Television Standards Committee Inc. 2001.

A copy of this standard is available at <http://www.atsc.org>.

Table 10.24 Number of quantizer levels and mantissa bits.

bap	Quantizer levels	Mantissa bits (qntztab[bap])
0	0	0
1	3	5/3
2	5	7/3
3	7	3
4	11	7/2
5	15	4
6	32	5
7	64	6
8	128	7
9	256	8
10	512	9
11	1024	10
12	2048	11
13	4096	12
14	16384	14
15	65536	16

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.25 Mantissa code words for symmetric quantization (bap = 1).

mantissa_code	mantissa value
0	-2/3
1	0
2	2/3

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.26 Mantissa code words for symmetric quantization (bap = 2).

mantissa_code	mantissa value
0	$-4/5$
1	$-2/5$
2	0
3	$2/5$
4	$4/5$

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

For bap values of 1 and 2, three mantissa_code values are treated as unsigned integers and grouped together to form a single larger code word using the following equations:

bap = 1:

$$\text{group_code} = 9 \times \text{mantissa_code}[a] + 3 \times \text{mantissa_code}[b] + \text{mantissa_code}[c] \quad (10.19)$$

bap = 2:

$$\text{group_code} = 25 \times \text{mantissa_code}[a] + 5 \times \text{mantissa_code}[b] + \text{mantissa_code}[c] \quad (10.20)$$

For a bap value of 4, two mantissa_code values are treated as unsigned integers and grouped together to form a single larger code word using the following equation:

bap = 4:

$$\text{group_code} = 11 \times \text{mantissa_code}[a] + \text{mantissa_code}[b] \quad (10.21)$$

Table 10.27 Mantissa code words for symmetric quantization (bap = 3).

mantissa_code	mantissa value
0	$-6/7$
1	$-4/7$
2	$-2/7$
3	0
4	$2/7$
5	$4/7$
6	$6/7$

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.28 Mantissa code words for symmetric quantization (bap = 4).

mantissa_code	mantissa value
0	$-10/11$
1	$-8/11$
2	$-6/11$
3	$-4/11$
4	$-2/11$
5	0
6	$2/11$
7	$4/11$
8	$6/11$
9	$8/11$
10	$10/11$

© Advanced Television Standards Committee Inc. 2001.
A copy of this standard is available at <http://www.atsc.org>.

Table 10.29 Mantissa code words for symmetric quantization (bap = 5).

mantissa_code	mantissa value
0	$-14/15$
1	$-12/15$
2	$-10/15$
3	$-8/15$
4	$-6/15$
5	$-4/15$
6	$-2/15$
7	0
8	$2/15$
9	$4/15$
10	$6/15$
11	$8/15$
12	$10/15$
13	$12/15$
14	$14/15$

© Advanced Television Standards Committee Inc.
2001. A copy of this standard is available at <http://www.atsc.org>.

where index *a* refers to the `mantissa_code` value with the lowest frequency in the group and indexes *b* and *c* refer to values with successively higher frequencies.

Note that these `mantissa_code` values are not required to be immediately adjacent to each other in frequency. The `group_code` value is constructed from the next available `mantissa_code` values in order of ascending frequency. If the number of `mantissa_code` values in a channel is not a multiple of the group size, then the `mantissa_code` value from the next channel to be processed is used to complete the group. If the number of `mantissa_code` values in an audio block is not a multiple of the group size, then the last group in the block is completed by padding with dummy mantissa values.

10.1.9.2. *Asymmetric Quantization ($6 \leq \text{bap} \leq 15$)*

For `bap` values of 6 to 15, the quantized mantissa value is defined as the fractional 2's complement representation of the mantissa value. So, if the number of bits allocated to each mantissa is designated by `qntztab[bap]` and the decimal point is considered to be to the left of the MSB, the 2's complement form of the normalized mantissa can represent values in the range -1.0 to $(1.0 - 2^{-(\text{qntztab}[\text{bap}]-1)})$.

10.1.9.3. *Dither for Zero Bit Mantissas*

Dither is the term commonly used for the substitution of zero values with random noise. In the AC-3 decoder, those coefficient mantissas that have been allocated a `bap` of zero may be reconstructed using random noise. The use of dither is controlled by the single bit parameter **dithflag**. A value of 1 for **dithflag** indicates that dither should be used, and a value of 0 indicates that the coefficient should be reconstructed as a true zero value. A **dithflag** bit is transmitted for each channel, and the dither noise is added after individual channel coefficients have been extracted from the coupling channel. The dither noise is added in this way so that the noise in each channel remains uncorrelated.

The random noise sequence used to generate the dither values should have a uniform distribution and ideally has values in the range -0.707 to 0.707 . However, this range can be approximated to -0.75 to 0.75 or even -0.5 to 0.5 to reduce computational complexity.

10.1.10. **Dialog Normalization**

In some cases there may be a significant loudness difference for normal spoken dialog between different portions of a television broadcast. For example, the loudness level of dialog in a commercial may be different from that in the movie being broadcast at the time. The **dialnorm** parameter in the AC-3 bit stream provides a mechanism for alleviating this problem and allowing normal dialog from all parts of the broadcast signal to be reproduced at the same loudness level. The **dialnorm**

parameter is interpreted as a 5-bit unsigned integer that indicates the subjective level of spoken dialog relative to the maximum possible reproduced sound level. For example a **dialnorm** value of 25 indicates that the level of spoken dialog in the current transmission is at a level of 25 dB below the maximum output level of the decoder.

The **dialnorm** parameter is not intended to be used by the decoder itself but instead can be used in the section of the sound reproduction system that controls the output sound level. This section of the system typically uses **dialnorm** to adjust the output sound level so that spoken dialog is always reproduced at a constant volume level while allowing the maximum volume level to vary for different parts of the broadcast signal.

EXAMPLE 10.5

Consider the situation where the soundtrack of a movie is currently being reproduced with a **dialnorm** value of 20 and the listener has adjusted the desired output volume to 65 dB. The volume control section of the sound reproduction system adjusts the output volume so that the maximum decoded signal level is reproduced at 85 dB (i.e., 20 dB above the level for spoken dialog) and, hence, normal spoken dialog is reproduced at the desired 65 dB.

Now consider what happens if a commercial message is inserted into the broadcast and is transmitted with a **dialnorm** value of 10. The volume control section can use this value to automatically adjust the output volume so that the maximum decoded signal level is reproduced at 75 dB. Consequently, the spoken dialog in the commercial message is automatically normalized to 65 dB (10 dB below the maximum level) and is reproduced at the same output volume as the dialog in the movie. ■

10.1.11. Dynamic Range Compression

There is often a requirement for different listeners to be able to reproduce the same audio signal with different amounts of *dynamic range*. The original dynamic range for some audio signals can be very wide. For example a soundtrack for a feature film may be produced with loud sounds, such as explosions, that are 20 dB louder than normal dialog and quiet sounds, such as leaves rustling, that are 50 dB quieter than the level for dialog.

There are many listening situations where this amount of dynamic range is unacceptable, and it is desirable to reduce the volume of loud sounds and also increase the volume of quiet sounds. These limitations on the reproduced volume of signals are collectively known as *dynamic range compression*. However, there also situations where the listener would like to receive the signal with the original dynamic range.

To satisfy the requirements of both these situations, the AC-3 bit stream may contain a sequence of dynamic range control values. These control values are transmitted as 8-bit code words designated by **dynrng**. A **dynrng** code

word may be transmitted at the start of each audio block and is used to indicate the alteration to the original sound level required to implement dynamic range compression.

EXAMPLE 10.6

Consider the case of a soundtrack for a feature film that has an original dialog level of 25 dB less than the maximum level. In this soundtrack, very loud sounds can reach the maximum level of 25 dB greater than the dialog level and quiet sounds can reach 50 dB below the dialog level.

A typical set of **dynrng** parameters would indicate a reduction in level for those sounds above the dialog level and an increase in level for those sounds below the dialog level. The loudest sounds might be reduced by 15 dB, and the quietest sounds might be increased by 20 dB. Hence, these **dynrng** parameters provide a well-defined mechanism for compressing the reproduced dynamic range of the transmitted audio signal from 75 to 40 dB. Use of the **dynrng** parameters also allows the reproduction system to maintain a constant volume level for spoken dialog while reducing the volume of loud sounds, such as explosions, and increasing the volume of quiet sounds that would otherwise be inaudible. ■

The default action for the AC-3 decoder is to implement the gain control indicated by the **dynrng** parameters. However, users may optionally decide to fully or partially disable the dynamic range compression in order to reproduce the signal with some or all of its original dynamic range.

10.1.11.1. Detailed Implementation

The 8-bit **dynrng** parameter may be transmitted with any audio block. The presence of this parameter is indicated with the 1 bit parameter **dynrng**_e, with a value of 1 for **dynrng**_e indicating that a **dynrng** parameter follows in the bit stream. A value of 0 for **dynrng**_e in blocks 2 to 6 of an audio frame indicates that the **dynrng** parameter from the previous block should be used. For the first block in the audio frame, a value of 0 indicates that a **dynrng** code word of “00000000” should be used.

The first three bits of the **dynrng** code word are used to indicate a gain change that can be implemented using a simple left or right side shift. Table 10.30 shows the number of shifts indicated by the first three bits of the **dynrng** code word and the corresponding gain adjustment in dB.

The remaining 5 bits of the **dynrng** code word are interpreted as the five least significant bits of a 6-bit unsigned binary fraction with a leading value of 1. So the fractional values that can be represented in this way range from 0.11111_2 ($63/64$ or -0.14 dB) to 0.10000_2 ($1/2$ or -6.02 dB). These fractional gain adjustments can then be implemented using a 6-bit multiply operation.

The gain adjustments indicated by the two segments of the **dynrng** code word are consecutively implemented and, hence, allow for gain changes of $-18.06 - 6.02 = -24.08$ dB to $24.08 - 0.14 = 23.94$ dB in steps of 0.14 dB.

Table 10.30 Interpretation of the three most significant bits of **dynrng**.

Three most significant bits of dynrng	Number of shifts	Equivalent gain adjustment (dB)
011	4 left	24.08
010	3 left	18.06
001	2 left	12.04
000	1 left	6.02
111	none	0
110	1 right	−6.02
101	2 right	−12.04
100	3 right	−18.06

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.1.12. Heavy Compression

The AC-3 syntax provides a second means of transmitting dynamic range compression information in the BSI header of the audio frame. The 8-bit code word **compr** can be used to indicate a larger reduction in dynamic range than the **dynrng** code words in each audio block. The gain adjustment indicated by **compr** can also be used to guarantee that a monophonic downmix of the transmitted audio signal does not exceed a certain peak level.

The large dynamic range reduction (or *heavy compression*) provided by the **compr** control signal may be useful when the audio signal is to be delivered in a quiet environment such as a hotel room or an airline seat. The gain adjustment indicated by **compr** has twice the range of that indicated by the **dynrng** parameters and half the resolution (i.e., −48 to 48 dB rather than −24 to 24 dB in steps of 0.28 rather than 0.14 dB) but only has a time resolution of an audio frame rather than a block (i.e., 32 to 48 ms rather than 5.33 to 8 ms).

The peak downmix level limitation may be required when a monophonic downmix is to be used to modulate an RF signal and overmodulation is to be avoided. For example, a digital set-top decoder may be required to modulate video and audio signals onto an RF channel in order for the broadcast signal to be received by an analog television receiver.

User implementations that require the peak downmix level to be constrained should use the **compr** code words rather than the **dynrng** control signal. However, it may only be necessary to transmit **compr** parameters when the compression provided by **dynrng** does not adequately restrict the peak level. In such cases, if the decoder has been instructed to use **compr**, and the **compr** code word is not present in the bit stream, then the decoder should use the **dynrng** code words for peak level limitation.

10.1.12.1. Detailed Implementation

The 8-bit **compr** parameter may be transmitted with any audio frame. The presence of this parameter is indicated by the 1 bit parameter **compre**. A value of 1 for **compre** indicates that a **compr** parameter follows in the bit stream.

The first four bits of the **compr** code word are used to indicate a gain change that can be implemented using a simple left or right shift. Table 10.31 shows the number of shifts indicated by the first three bits of the **compr** code word and the corresponding gain adjustment in dB.

The remaining four bits of the **compr** code word are interpreted as the four least significant bits of a 5-bit unsigned binary fraction with a leading value of 1. So the fractional values that can be represented in this way range from 0.11111_2 (31/32 or -0.28 dB) to 0.10000_2 (1/2 or -6.02 dB). These fractional gain adjustments can then be implemented using a 5-bit multiply operation.

The gain adjustments indicated by the two segments of the **compr** code word are consecutively implemented and hence allow for gain changes of $-42.14 - 6.02 = -48.16$ dB to $48.16 - 0.28 = 47.88$ dB in steps of 0.28 dB.

10.1.13. Downmixing

If the number of speakers in the reproduction system does not match the number of channels provided in the transmitted audio signal, then some form of *downmixing*

Table 10.31 Interpretation of the four most significant bits of **compr**.

Four most significant bits of compr	Number of shifts	Equivalent gain adjustment (dB)
0111	8 left	48.16
0110	7 left	42.14
0101	6 left	36.12
0100	5 left	30.10
0011	4 left	24.08
0010	3 left	18.06
0001	2 left	12.04
0000	1 left	6.02
1111	none	0
1110	1 right	-6.02
1101	2 right	-12.04
1100	3 right	-18.06
1011	4 right	-24.08
1010	5 right	-30.10
1001	6 right	-36.12
1000	7 right	-42.14

is required. The downmixing procedure for an AC-3 decoder is standardized so that audio content providers can be certain of the final audio quality of systems with various loudspeaker configurations.

10.1.13.1. General Downmix Procedure.

The unnormalized downmix coefficients for a range of channel and speaker configurations can be determined using the following pseudocode. Once these coefficients have been determined, it may be necessary to apply a normalization procedure to prevent arithmetic overflow. This normalization is achieved by attenuating all the downmix coefficients equally so that the sum of the coefficients used to create any single output channel does not exceed 1.0.

Pseudocode

```
downmix()
{
  if (acmod == 0) /* 1+1 mode, dual independent mono channels present */
  {
    if (output_nfront == 1) /* 1 front loudspeaker (center) */
    {
      if (dualmode == Chan 1) /* Ch1 output requested */
      {
        route left into center ;
      }
      else if (dualmode == Chan 2) /* Ch2 output requested */
      {
        route right into center ;
      }
      else
      {
        mix left into center with -6 dB gain ;
        mix right into center with -6 dB gain ;
      }
    }
  }
  else if (output_nfront == 2) /* 2 front loudspeakers (left, right) */
  {
    if (dualmode == Stereo)
      /* output of both mono channels requested */
    {
      route left into left ;
      route right into right ;
    }
    else if (dualmode == Chan 1)
    {
      mix left into left with -3 dB gain ;
      mix left into right with -3 dB gain ;
    }
  }
}
```

```

else if (dualmode == Chan 2)
{
    mix right into left with -3 dB gain ;
    mix right into right with -3 dB gain ;
}
else /* mono sum of both mono channels requested */
{
    mix left into left with -6 dB gain ;
    mix right into left with -6 dB gain ;
    mix left into right with -6 dB gain ;
    mix right into right with -6 dB gain ;
}
}
else /* output_nfront == 3 */
{
    if (dualmode == Stereo)
    {
        route left into left ;
        route right into right ;
    }
    else if (dualmode == Chan 1)
    {
        route left into center ;
    }
    else if (dualmode == Chan 2)
    {
        route right into center ;
    }
    else
    {
        mix left into center with -6 dB gain ;
        mix right into center with -6 dB gain ;
    }
}
}
else /* acmod > 0 */
{
    for i = { left, center, right, leftsur/monosur, rightsur }
    {
        if (exists(input_chan[i])) and (exists(output_chan[i]))
        {
            route input_chan[i] into output_chan[i] ;
        }
    }
    if (output_mode == 2/0 Dolby Surround compatible)
    /* 2 ch matrix encoded output requested */
    {
        if (input_nfront != 2)
        {

```

```

        mix center into left with -3 dB gain ;
        mix center into right with -3 dB gain ;
    }
    if (input_nrear == 1)
    {
        mix -mono surround into left with -3 dB gain ;
        mix mono surround into right with -3 dB gain ;
    }
    else if (input_nrear == 2)
    {
        mix -left surround into left with -3 dB gain ;
        mix -right surround into left with -3 dB gain ;
        mix left surround into right with -3 dB gain ;
        mix right surround into right with -3 dB gain ;
    }
}
else if (output_mode == 1/0) /* center only */
{
    if (input_nfront != 1)
    {
        mix left into center with -3 dB gain ;
        mix right into center with -3 dB gain ;
    }
    if (input_nfront == 3)
    {
        mix center into center using clev and +3 dB gain ;
    }
    if (input_nrear == 1)
    {
        mix mono surround into center using slev;
    }
    else if (input_nrear == 2)
    {
        mix left surround into center using slev and -3 dB gain ;
        mix right surround into center using slev and -3 dB gain ;
    }
}
else /* more than center output requested */
{
    if (output_nfront == 2)
    {
        if (input_nfront == 1)
        {
            mix center into left with -3 dB gain ;
            mix center into right with -3 dB gain ;
        }
        else if (input_nfront == 3)
        {
            mix center into left using clev ;

```

```

        mix center into right using clef ;
    }
}
if (input_nrear == 1) /* single surround channel coded */
{
    if (output_nrear == 0) /* no surround loudspeakers */
    {
        mix mono surround into left with slef and -3 dB gain ;
        mix mono surround into right with slef and -3 dB gain ;
    }
    else if (output_nrear == 2) /* two surround loudspeaker channels */
    {
        mix mono srnd into left surround with -3 dB gain ;
        mix mono srnd into right surround with -3 dB gain ;
    }
}
else if (input_nrear == 2) /* two surround channels encoded */
{
    if (output_nrear == 0)
    {
        mix left surround into left using slef ;
        mix right surround into right using slef ;
    }
    else if (output_nrear == 1)
    {
        mix left srnd into mono surround with -3 dB gain ;
        mix right srnd into mono surround with -3 dB gain ;
    }
}
}
}
}

```

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

The downmix coefficients **clef** and **slef** are indicated by the 2-bit code words **cmixlef** and **surmixlef**, respectively. These code words are transmitted in the BSI part of the audio frame, and the coefficient value corresponding to each code word is shown in Table 10.32. If the reserved code is received for **cmixlef**, the decoder should use the intermediate value of 0.596. If the reserved code is received for **surmixlef**, the decoder should use the intermediate value of 0.5.

10.1.13.2. Downmixing to Stereo

If surround channels are transmitted and the output required is a stereo pair, then two types of downmix are required. The first is a downmix to a conventional stereo

Table 10.32 Center and surround mix levels.

cmixlev	clev	surmixlev	slev
00	0.707	00	0.707
01	0.596	01	0.5
10	0.5	10	0
11	Reserved	11	Reserved

© Advanced Television Standards Committee Inc. 2001.

A copy of this standard is available at <http://www.atsc.org>.

pair, L_o and R_o , and the second is a downmix to a matrix surround encoded stereo pair, L_t and R_t .

If all five channels are transmitted (3/2 mode) and the output required is a conventional stereo pair, then the following downmix equations are to be used.

$$L_o = L + \text{clev} \times C + \text{slev} \times L_s \quad (10.22)$$

$$R_o = R + \text{clev} \times C + \text{slev} \times R_s \quad (10.23)$$

If the output required is a matrix surround encoded stereo pair, then the following downmix equations are to be used.

$$L_t = L + 0.707 \times (C - L_s - R_s) \quad (10.24)$$

$$R_t = R + 0.707 \times (C + L_s + R_s) \quad (10.25)$$

If the center channel is not transmitted (2/2 mode), the same equations can be used without the C term.

If only a single surround channel is transmitted (3/1 mode) and the output required is a conventional stereo pair, then the following downmix equations are to be used.

$$L_o = L + \text{clev} \times C + 0.707 \times \text{slev} \times S \quad (10.26)$$

$$R_o = R + \text{clev} \times C + 0.707 \times \text{slev} \times S \quad (10.27)$$

If the output required is a matrix surround encoded stereo pair, then the following downmix equations are to be used.

$$L_t = L + 0.707 \times (C - S) \quad (10.28)$$

$$R_t = R + 0.707 \times (C + S) \quad (10.29)$$

If the center channel is not transmitted (2/1 mode), the same equations can be used without the C term. If the surround channels are not transmitted (3/0 mode), the same equations can be used without the L_s , R_s , or S terms.

10.1.13.3. Downmixing to Mono

If the output required is a monophonic signal, the downmixed stereo signals may be further downmixed to mono using a simple summation of the two channels. However, if the matrix surround encoded pair is used, the surround information is lost, so it is desirable to use the conventional stereo pair.

If the monophonic signal is to be calculated directly and all channels are transmitted, the following equation is to be used.

$$M = L + R + 2.0 \times \text{clev} \times C + \text{slev} \times (L_s + R_s) \tag{10.30}$$

If the center channel is not transmitted (2/2 mode), the same equation can be used without the C term. If only a single surround channel is transmitted (3/1 mode), the following equation is to be used.

$$M = L + R + 2.0 \times \text{clev} \times C + 1.4 \times \text{slev} \times S \tag{10.31}$$

If the center or surround channels are not transmitted (2/1, 3/0, or 2/0 mode), the same equation may be used without the C or S terms.

10.1.13.4. Normalizing Downmixing Coefficients

If all the channels to be included in a downmixed signal have their maximum value at the same time, the values in the signal will exceed the maximum available word-length. This problem is known as *arithmetic overflow* and can be avoided if the coefficients used in the downmixing equations described in the previous section are scaled downwards. A different scaling could be applied for all possible combinations of input and output channels, but this would require a very large number of scaled coefficients to be defined. Instead, the worst-case situation is used to determine the maximum scaling factor required when clev and slev are both equal to 0.707.

For a conventional stereo output, if unscaled coefficients are used, the maximum downmixed signal has a value of $1 + 0.707 + 0.707 = 2.414$. So the maximum scaling required for the coefficients is $1/2.414 = 0.4143$. Table 10.33 shows the unscaled and scaled versions of all the possible downmixing coefficients required to produce a conventional stereo pair. The downmixing equation is typically implemented using

Table 10.33 Scaled downmix coefficients for a conventional stereo signal.

Unscaled coefficient	Scaled coefficient	6-bit Quantized coefficient
1.0	0.414	26/64
0.707	0.293	18/64
0.596	0.247	15/64
0.5	0.207	13/64
0.354	0.147	9/64

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.34 Scaled downmix coefficients for a matrix surround encoded stereo signal.

Unscaled coefficient	Scaled coefficient	6-bit Quantized coefficient
1.0	0.3204	26/64
0.707	0.293	18/64

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

fixed length binary arithmetic, and if this is the case the 6-bit quantized coefficients shown in Table 10.33 produce a downmixed signal with the required accuracy. If a mono signal is required, the coefficients need to be scaled by a further factor of 0.5.

For a matrix surround encoded stereo output, if unscaled coefficients are used, the maximum downmixed signal has a value of $1 + 0.707 + 0.707 + 0.707 = 3.121$. So the maximum scaling required for the coefficients is $1/3.121 = 0.3204$. Table 10.34 shows the unscaled, scaled, and 6-bit quantized versions of all the possible down-mixing coefficients required to produce a matrix surround encoded stereo pair.

10.2. SYNTAX

The syntax of the AC-3 bit stream is specified in this section. The order of arrival of the bits is defined using the methodology described in Chapter 5. If the bit-stream element to be transmitted is larger than 1 bit, it is transmitted in one of two ways: (a) if the element specifies a numerical value, it is transmitted with the most significant bit first and (b) if the element is a bit field, it is transmitted with the left bit first. In some cases, the meaning for a possible value of a bit-stream element is “reserved.” If the decoder erroneously receives a reserved value for an element, there are two possible responses for the decoder: (a) the decoder cannot decode any value and (b) the decoder can use a default value for the element.

10.2.1. Syntax Specification

The structure of an AC-3 synchronization frame is shown in Figure 10.2. The synchronization frame begins with a syncinfo (SI) header. The bit-stream information (BSI) header is transmitted next, and this is followed by six audio blocks (AB). Each audio block contains the information required to decode 256 audio samples. The synchronization frame may then contain an optional auxiliary data field (Aux) and ends with an error check field that contains a cyclic redundancy code word (CRC) used for error detection. The syntax specification for an AC-3 bit stream is given in Table 10.35. The bit stream consists of consecutive synchronization frames.

Table 10.35 Syntax specification for an AC-3 bit stream.

Syntax	Bits
AC-3_bit stream() { while(true) { syncframe(); } }/* end of AC-3 bit stream */	

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

The syntax specification for a syncframe is given in Table 10.36.
Each field of the syncframe is described in the following sections.

10.2.1.1. Synchronization Information (syncinfo)

The syncinfo field contains information necessary for the decoder to maintain synchronization of the incoming bit stream. The syntax specification of the syncinfo field is given in Table 10.37.

A description of the bit stream elements contained in the syncinfo field is given in Table 10.38.

10.2.1.2. Bit-Stream Information (bsi)

The BSI field contains parameters that specify aspects of the audio service being transmitted. The syntax specification of the BSI field is given in Table 10.39.

Table 10.36 Syntax specification for a syncframe.

Syntax	Bits
<pre>syncframe() { syncinfo(); bsi(); for(blk = 0; blk < 6; blk++) { audblk(); } auxdata(); errorcheck(); }/* end of syncframe */</pre>	

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

Table 10.37 Syntax specification for a syncinfo field.

Syntax	Bits
syncinfo() { syncword ; crc1 ; fscod ; frmsizecod ; }/* end of syncinfo */	 16 16 2 6

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

A description of the bit stream elements contained in a BSI field is given in Table 10.40.

10.2.1.3. Audio Block (audblk)

Each audio block contains the information required to decode 256 audio samples. The syntax specification of the audblk field is given in Table 10.41.

A description of the bit stream elements contained in an audblk field is given in Table 10.42.

Table 10.38 Description of the bit stream elements in a syncinfo field.

Element	Description
syncword	Used to locate the start of the syncframe (the value of the syncword is always 0x0B77)
crc1	Error check bits
fscod	Specifies the input sampling rate (see Table 10.1)
frmsizecod	Specifies the number of 16-bit words in the current syncframe (see Ref. [1])

Table 10.39 Syntax specification of the BSI field.

Syntax	Bits
bsi() { bsid bsmod acmod if((acmod & 0x1) && (acmod != 0x1))/* if 3 front channels */{ cmixlev } if(acmod & 0x4)/* if a surround channel exists */{ surmixlev }	 5 3 3 2 2

(continued)

Table 10.39 (Continued)

Syntax	Bits
if(acmod == 0x2) /* if in 2/0 mode */ { dsurmod }	2
lfeon	1
dialnorm	5
compre	1
if(compre) { compr }	8
langcode	1
if(langcode) { langcod }	8
audprodie	1
if(audprodie)	
{	
mixlevel	5
roomtyp	2
}	
if(acmod == 0) /* if 1+1 mode (dual mono, so some items need a second value) */	
{	
dialnorm2	5
compr2e	1
if(compr2e) { compr2 }	8
lngcod2e	1
if(lngcod2e) { langcod2 }	8
audprodi2e	1
if(audprodi2e)	
{	
mixlevel2	5
roomtyp2	2
}	
}	
copyrightb	1
origbs	1
timecod1e	1
if(timecod1e) { timecod1 }	14
timecod2e	1
if(timecod2e) { timecod2 }	14
addbsie	1
if(addbsie)	
{	
addbsil	6
addbsi (addbsil+1)	8
}	
} /* end of bsi */	

Table 10.40 Description of the bit stream elements in a BSI field.

Element	Description
bsid	Specifies the version of the standard (currently 8).
bsmod	Specifies the type of service that this audio bit stream is providing (see Ref. [1]).
acmod	Specifies the channel configuration in use (see Table 10.2).
cmixlev	Specifies the downmix level for the center channel relative to the left and right side channels (see Ref. [1]).
surmixlev	Specifies the downmix level for the surround channels relative to the left and right side channels (see Ref. [1]).
dsurmod	Specifies whether the program has been encoded using Dolby surround mode (see Ref. [1]).
lfeon	A value of “1” indicates that the low-frequency effects (LFE) channel is in use.
dialnorm	Specifies how far the average dialog level is below the maximum level (see Section 10.1.10).
compre	A value of “1” indicates that a compression gain word follows in the bit stream.
compr	The compression gain word (see Section 10.1.12).
langcode	A value of “1” indicates that a language code follows in the bit stream.
langcod	The language code word (see Ref. [1]).
audprodi	A value of “1” indicates that audio production information follows in the bit stream.
mixlevel	Specifies the absolute sound pressure level of a channel (see Ref. [1]).
roomtyp	Specifies the type of room used in the final mixing process (see Ref. [1]).
dialnorm2	Used for channel 2 in dual mono mode.
compr2e	Used for channel 2 in dual mono mode.
compr2	Used for channel 2 in dual mono mode.
langcod2e	Used for channel 2 in dual mono mode.
langcod2	Used for channel 2 in dual mono mode.
audprodi2e	Used for channel 2 in dual mono mode.
mixlevel2	Used for channel 2 in dual mono mode.
roomtyp2	Used for channel 2 in dual mono mode.
copyrightb	A value of “1” indicates that the following bit stream is protected by copyright.
origbs	A value of “1” indicates that the following bit stream is an original bit stream.
timecod1e	A value of “1” indicates that the first half of a time code follows in the bit stream.
timecod1	Specifies a time with resolution of 8 s and full scale of 24 h (see Ref. [1]).
timecod2e	A value of “1” indicates that the second half of a time code follows in the bit stream.
timecod2	Specifies a time with resolution of 1/64th of a frame and full scale of 8 s (see Ref. [1]).
addbsie	A value of “1” indicates that additional information follows in the bit stream.
addbsil	Specifies the length in bytes of the additional information that follows.
addbsi	Additional information.

Table 10.41 Syntax specification of the audblk field.

Syntax	Bits
audblk()	
{	
/* These fields for block switch and dither flags */	
for(ch = 0; ch < nfchans; ch++) { blksw[ch] }	1
for(ch = 0; ch < nfchans; ch++) { dithflag[ch] }	1
/* These fields for dynamic range control */	
dynrng	1
if(dynrng) { dynrng }	8
if(acmod == 0) /* if 1+1 mode */	
{	
dynrng2e	1
if(dynrng2e) { dynrng2 }	8
}	
/* These fields for coupling strategy information */	
cplstre	1
if(cplstre)	
{	
cplinu	1
if(cplinu)	
{	
for(ch = 0; ch < nfchans; ch++) { chincpl[ch] }	1
if(acmod == 0x2) { phsflginu } /* if in 2/0 mode */	1
cplbegf	4
cplendf	4
/* ncplsubnd = 3 + cplendf - cplbegf */	
for(bnd = 1; bnd < ncplsubnd; bnd++) { cplbndstre[bnd] }	1
}	
}	
/* These fields for coupling coordinates, phase flags */	
if(cplinu)	
{	
for(ch = 0; ch < nfchans; ch++)	
{	
if(chincpl[ch])	
{	
cplcoe[ch]	1
if(cplcoe[ch])	
{	
mstreplco[ch]	2
/* ncplbnd derived from ncplsubnd, and cplbndstre */	
for(bnd = 0; bnd < ncplbnd; bnd++)	
{	
cplcoexp[ch][bnd]	4
}	
}	
}	

Table 10.41 (Continued)

Syntax	Bits
cplcomant[ch][bnd]	4
<pre> } } } if((acmod==0x2)&&phsf!ginu&&(cplcoe[0] cplcoe[1])) { for(bnd = 0; bnd < ncplbnd; bnd++) {phsflg[bnd]} } } /* These fields for rematrixing operation in the 2/0 mode */ if(acmod == 0x2) /* if in 2/0 mode */ { rematstr if(rematstr) { if((cplbegf > 2) (cplinu == 0)) { for(rbnd = 0; rbnd < 4; rbnd++) {rematflg[rbnd]} if((2 >= cplbegf > 0) && cplinu) { for(rbnd = 0; rbnd < 3; rbnd++) {rematflg[rbnd]} } if((cplbegf == 0) && cplinu) { for(rbnd = 0; rbnd < 2; rbnd++) {rematflg[rbnd]} } } } } /* These fields for exponent strategy */ if(cplinu) {cplexpstr} for(ch = 0; ch < nfchans; ch++) {chexpstr[ch]} if(lfeon) {lfeexpstr} for(ch = 0; ch < nfchans; ch++) { if(chexpstr[ch] != reuse) { if(!chincpl[ch]) {chbwcod[ch]} } } } /* These fields for exponents */ if(cplinu) /* exponents for the coupling channel */ { </pre>	1 1 1 1 1 2 2 1 6

(continued)

Table 10.41 (Continued)

Syntax	Bits
if(cplexpstr != reuse)	
{	
cplabsexp	4
/* ncplgrps derived from ncplsubnd, cplexpstr */	
for(grp = 0; grp < ncplgrps; grp++) { cplexps[grp] }	7
}	
for(ch = 0; ch < nfchans; ch++) /* exponents for full bandwidth channels */	
{	
if(chexpstr[ch] != reuse)	
{	
exps[ch][0]	4
/* nchgrps derived from chexpstr[ch], and cplbegf or chbwcod[ch] */	
for(grp = 1; grp <= nchgrps[ch]; grp++) { exps[ch][grp] }	7
gainrng[ch]	2
}	
}	
if(lfeon) /* exponents for the low-frequency effects channel */	
{	
if(lfeexpstr != reuse)	
{	
lfeexps[0]	4
/* nlfegrps = 2 */	
for(grp = 1; grp <= nlfegrps; grp++) { lfeexps[grp] }	7
}	
}	
/* These fields for bit-allocation parametric information */	
baie	1
if(baie)	
{	
sdccycod	2
fdccycod	2
sgaincod	2
dbpbcod	2
floorcod	3
}	
snroffste	1
if(snroffste)	
{	
csnroffst	6
if(cplinu)	
{	
cplfsnroffst	4
cplfgaincod	3
}	

Table 10.41 (Continued)

Syntax	Bits
<pre> } for(ch = 0; ch < nfchans; ch++) { fsnroffst[ch] fgaincod[ch] } if(lfeon) { lfefsnroffst lfefgaincod } } if(cplinu) { cplleake if(cplleake) { cpflf leak cppls leak } } /* These fields for delta bit-allocation information */ deltbaie if(deltbaie) { if(cplinu) {cpldeltbae} for(ch = 0; ch < nfchans; ch++) {deltbae[ch]} if(cplinu) { if(cpldeltbae==new info follows) { cpldelt nseg for(seg = 0; seg <= cpldelt nseg; seg++) { cpldelt offst[seg] cpldelt len[seg] cpldelt ba[seg] } } } for(ch = 0; ch < nfchans; ch++) { </pre>	<pre> 4 3 4 3 1 3 3 1 2 2 3 5 4 3 </pre>

(continued)

Table 10.41 (Continued)

Syntax	Bits
if(deltbae[ch]==new info follows) { deltalseg[ch] for(seg= 0; seg <= deltnseg[ch]; seg++) { deltfst[ch][seg] deltlen[ch][seg] deltba[ch][seg] } } }	3 5 4 3
/* These fields for inclusion of unused dummy data */ skiple if(skiple) { skipl skipfld }	1 9 skipl × 8
/* These fields for quantized mantissa values */ got_cplchan = 0 for (ch = 0; ch < nfchans; ch++) { for (bin = 0; bin < nchmant[ch]; bin++) { chmant[ch][bin] } if (cplinu && chincpl[ch] && !got_cplchan) { for (bin = 0; bin < ncplmant; bin++) { cplmant[bin] } got_cplchan = 1 } } if(lfeon)/* mantissas of low-frequency effects channel */ { for (bin = 0; bin < nlfemant; bin++) { lfemant[bin] } } } /* end of audblk */	(0 – 16) (0 – 16)

Table 10.42 Description of the bit stream elements in an audblk field.

Element	Description
blksw[ch]	A value of “1” indicates that the current audio block was coded using two blocks in channel ch (see Section 10.1.2).
dithflag[ch]	A value of “1” indicates that the decoder should use dither in channel ch (see Section 10.1.9.3).
dynrng	A value of “1” indicates that a dynamic range gain word follows in the bit stream.
dynrng	The dynamic range gain word (see Section 10.1.11).
dynrng2e	Used for channel 2 in dual mono mode.
dynrng2	Used for channel 2 in dual mono mode.
cplstre	A value of “1” indicates that new coupling information follows.
cplinu	A value of “1” indicates that coupling is in use.
chincpl[ch]	A value of “1” indicates that channel ch is coupled.
phsflginu	A value of “1” indicates that phase flags are included with the coupling information.
cplbegf	Specifies the lowest frequency subband included in coupling (see Section 10.1.4).
cplendf	Specifies the highest frequency subband included in coupling (see Section 10.1.4).
cplbndstrc[sbnd]	A value of “1” indicates that the corresponding subband should be combined with the subband immediately below it in frequency (see Section 10.1.4).
cplcoe[ch]	A value of “1” indicates that coupling coordinates for channel ch follow (see Section 10.1.4).
mstrcplco[ch]	Specifies the master coupling coordinate gain factor for channel ch (see Section 10.1.4).
cplcoexp[ch][bnd]	The coupling coordinate exponent (see Section 10.1.4).
cplcomant[ch][bnd]	The coupling coordinate mantissa (see Section 10.1.4).
phsflg[bnd]	A value of “1” indicates that the coefficients of the right channel in coupling band bnd were negated before the coupling channel was formed (see Section 10.1.4).
rematstr	A value of “1” indicates that new rematrix flags follow.
rematflg[rband]	A value of “1” indicates that the coefficients in rematrixing band rband have been rematrixed (see Section 10.1.5).
cplexpstr	Specifies the method used to code exponents in the coupling channel (see Table 10.12).
chexpstr[ch]	Specifies the method used to code exponents in the full bandwidth channel ch (see Table 10.12).
lfeexpstr	Specifies the method used to code exponents in the LFE channel (see Section 10.1.6).
chbwcod[ch]	Specifies the highest frequency band included in the full bandwidth channel ch (see Section 10.1.6).
cplabsexp	The coupling absolute exponent (see Section 10.1.7).

(continued)

Table 10.42 *(Continued)*

Element	Description
cplexps[grp]	Specifies the value of differentially coded exponents for the coupling exponent group grp (see Section 10.1.7).
exps[ch][grp]	Specifies the encoded exponents for channel ch in group grp (see Section 10.1.7).
gainrng[ch]	Specifies an increase in dynamic range for channel ch (see Ref. [1]).
lfeexps[grp]	Specifies the encoded exponents for the LFE channel in group grp (see Section 10.1.7).
baie	A value of “1” indicates that bit-allocation information follows.
sdcycod	Specifies the slow delay parameter sdecay (see Section 10.1.8).
fdcycod	Specifies the fast-delay parameter fdecay (see Section 10.1.8).
sgaincod	Specifies the slow-gain parameter sgain (see Section 10.1.8).
dbpbcod	Specifies the parameter dbknee (see Section 10.1.8).
floorcod	Specifies the parameter floor (see Section 10.1.8).
snroffste	A value of “1” indicates that SNR offset parameters follow.
csnroffst	Specifies the coarse SNR offset (see Section 10.1.8).
cplfsnroffst	Specifies the fine SNR offset for the coupling channel (see Section 10.1.8).
cplfgaincod	Specifies the fast gain parameter fgain for the coupling channel (see Section 10.1.8).
fsnroffst[ch]	Specifies the fine SNR offset for the channel ch (see Section 10.1.8).
fgaincod[ch]	Specifies the fast gain parameter fgain for the channel ch (see Section 10.1.8).
lfefsnroffst	Specifies the fine SNR offset for the LFE channel (see Section 10.1.8).
lfefgaincod	Specifies the fast gain parameter fgain for the LFE channel (see Section 10.1.8).
cplleake	A value of “1” indicates that leak initialization parameters follow (see Section 10.1.8).
cplfleak	Specifies the parameter fastleak for the coupling channel (see Section 10.1.8).
cplslowleak	Specifies the parameter slowleak for the coupling channel (see Section 10.1.8).
deltbaie	A value of “1” indicates that delta bit allocation follows.
cpldeltbae	Specifies the delta bit-allocation mode for the coupling channel (see Section 10.1.8.7).
cpldeltalseg	Specifies the number of delta bit-allocation segments in the coupling channel (see Section 10.1.8.7).
cpldeltfst[seg]	Specifies the offset from the last band of the previous segment to the first band of the segment seg in the coupling channel (see Section 10.1.8.7).

Table 10.42 (Continued)

Element	Description
cpldeltlen[seg]	Specifies the number of bands in the segment seg in the coupling channel (see Section 10.1.8.7).
cpldeltba[seg]	Specifies the adjustment to the default masking curve in the segment seg in the coupling channel (see Section 10.1.8.7).
deltbae[ch]	Specifies the delta bit-allocation mode for channel ch (see Section 10.1.8.7).
deltntseg[ch]	Specifies the number of delta bit-allocation segments in channel ch (see Section 10.1.8.7).
deltfst[ch][seg]	Specifies the offset from the last band of the previous segment to the first band of the segment seg in channel ch (see Section 10.1.8.7).
deltlen[ch][seg]	Specifies the number of bands in the segment seg in channel ch (see Section 10.1.8.7).
deltba[ch][seg]	Specifies the adjustment to the default masking curve in the segment seg in channel ch (see Section 10.1.8.7).
skiple	A value of “1” indicates that padding bytes follow.
skipl	Specifies the number of padding bytes that follow.
skipfld	The padding bytes.
chmant[ch][bin]	The quantized mantissa value for frequency index bin in channel ch (see Section 10.1.9).
cplmant[bin]	The quantized mantissa value for frequency index bin in the coupling channel (see Section 10.1.9).
lfemant[bin]	The quantized mantissa value for frequency index bin in the LFE channel (see Section 10.1.9).

10.2.1.4. Auxiliary Data (auxdata)

User information may be placed at the end of a synchronization frame in the auxiliary data field. The syntax specification of the auxdata field is given in Table 10.43.

The auxiliary data is placed in the auxdata field so that an auxiliary data decoder can begin decoding from the end of the field. The auxiliary data decoder first examines the last bit of the auxdata field. This bit contains the flag **auxdatae**. If the value of **auxdatae** is “1,” the data decoder examines the previous 14 bits that contain the **auxdatal** code word. This code word specifies the length in bits of the user data contained in the auxdata field. The decoder can then back up **auxdatal** bits from the end of the **auxbits** field and begin decoding the auxiliary data. Note that the length of the auxbits field, **nauxbits**, may be greater than **auxdatal**. The auxiliary data field is structured in this way so that the auxiliary data can be decoded independently without having to first decode the audio data contained in the synchronization frame.

Table 10.43 Syntax specification of the auxdata field.

Syntax	Bits
auxdata() { auxbits if(auxdatae) { auxdatal } auxdatae } /* end of auxdata */	 nauxbits 14 1

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

10.2.1.5. Frame Error Detection Field (Error Check)

The syntax specification of the error check field is given in Table 10.44.

The one bit code word **cr crsv** is reserved for use in certain situations to guarantee that **crc2** is not equal to the syncword. The details of the error checking procedure using the 16 bit CRC code word **crc2** can be found in Ref. [1].

10.3. DECODER

10.3.1. Decode Exponents

The number of exponents to decode for each channel is determined from the bit-stream parameters **chbwcod[ch]** if the channel is not included in coupling and **cplbegf** and **cplendf** for the coupled and coupling channels. The starting bin number for the full bandwidth channels is always 0, that is $\text{fbwstrtmant}[\text{ch}] = 0$. The starting bin number for the coupling channel is given by

$$\text{cplstrtmant} = (\text{cplbegf} \times 12) + 37 \tag{10.32}$$

Table 10.44 Syntax specification of the error check field.

Syntax	Bits
errorcheck() { cr crsv crc2 } /* end of errorcheck */	 1 16

© Advanced Television Standards Committee Inc. 2001. A copy of this standard is available at <http://www.atsc.org>.

The last mantissa bin number for the full bandwidth channels is defined by

$$\text{fbwendmant}[\text{ch}] = ((\text{chbwcod}[\text{ch}] + 12) \times 3) + 37 \quad (10.33)$$

and for the coupling channel is defined by

$$\text{cplendmant} = ((\text{cplendf} + 3) \times 12) + 37 \quad (10.34)$$

For the low-frequency effects channel the start and end mantissa numbers are always given by $\text{lfestrtmant} = 0$ and $\text{lfeendmant} = 7$, respectively.

The number of grouped exponents to decode depends on the exponent strategy and (for independent and coupled channels) is given by the following equations.

D-15 mode:

$$\text{nchgrps}[\text{ch}] = \text{truncate}((\text{endmant}[\text{ch}] - 1)/3) \quad (10.35)$$

D-25 mode:

$$\text{nchgrps}[\text{ch}] = \text{truncate}((\text{endmant}[\text{ch}] - 1 + 3)/6) \quad (10.36)$$

D-45 mode:

$$\text{nchgrps}[\text{ch}] = \text{truncate}((\text{endmant}[\text{ch}] - 1 + 9)/12) \quad (10.37)$$

For the coupling channel, the number of grouped exponents to decode is given by the following equations.

D-15 mode:

$$\text{ncplgrps} = (\text{cplendmant} - \text{cplstrtmant})/3 \quad (10.38)$$

D-25 mode:

$$\text{ncplgrps} = (\text{cplendmant} - \text{cplstrtmant})/6 \quad (10.39)$$

D-45 mode:

$$\text{ncplgrps} = (\text{cplendmant} - \text{cplstrtmant})/12 \quad (10.40)$$

For the low-frequency effects channel, the number of groups is always given by $\text{nlfegrps} = 2$.

Then to obtain the set of 5-bit absolute exponents for each coefficient bin number the following procedure is used:

1. Each 7-bit grouped value, G , is decoded into three mapped values using the following equations (where $\%$ denotes the modulus operation):

$$\begin{aligned} M_1 &= \text{truncate}(G/25) \\ M_2 &= \text{truncate}((G \% 25)/5) \\ M_3 &= (G \% 25) \% 5 \end{aligned} \quad (10.41)$$

2. Each mapped value is converted to a differential exponent by subtracting a factor of 2.
3. The set of differential exponents is converted to a set of absolute exponents by adding each differential exponent to the previous absolute exponent value.
4. For the D-25 and D-45 modes, each absolute exponent is copied to the corresponding two or four coefficient bins.

10.3.2. Bit Allocation

The bit-allocation procedure is the same as that defined in Section 10.1.8., except that the value for `snroffset` is calculated using the transmitted values of the global coarse offset parameter, `csnroffset`, and the fine offset parameter for each full bandwidth channel, `fsnroffset[ch]`, the coupling channel, `cplfsnroffset`, and the low frequency channel, `lfefsnroffset`.

The setup of the psychoacoustic model is determined from the parameters transmitted in the bit stream so the following MATLAB code defines the initialization of the model.

For all channels:

```
sdecay = slowdec(sdcycod+1);
fdecay = fastdec(fdcycod+1);
sgain = slowgain(sgaincod+1);
dbknee = dbpbtabs(dbpbcod+1);
snrfloor = floortabs(floorcod+1);
```

For the full bandwidth channels:

```
strtmant = fbwstrtmant(ch);
endmant = fbwendmant(ch);
lowcomp = 0 ;
fgain = fastgain(fgaincod(ch)+1);
snroffset(ch) = (((csnroffset - 15)*16) + fsnroffset(ch))*4;
```

For the coupling channel:

```
startmant = cplstrtmant;
endmant = cplendmant;
fgain = fastgain(cplfgaincod+1);
snroffset = (((csnroffset - 15)*16) + cplfsnroffset)*4;
fastleak = (cplfleak*256) + 768 ;
slowleak = (cplsleak*256) + 768 ;
```

For the LFE channel:

```
strtmant = lfestrmant;
endmant = lfeendmant;
lowcomp = 0;
fgain = fastgain(lfefgaincod+1);
snroffset = (((csnroffset - 15)*16) + lfefsnroffset)*4;
```

10.3.3. Decode Coefficients

For each coefficient that is not included in coupling, a fixed point value is obtained by right shifting the mantissa value by its exponent as follows:

$$\text{coefficient}[\text{bin}] = \text{mantissa}[\text{bin}] \gg \text{exponent}[\text{bin}] \quad (10.42)$$

The value for mantissa[bin] depends on the type of quantization used and is determined using one of the following procedures.

10.3.3.1. Symmetric Quantization ($1 \leq \text{bap} \leq 5$)

For bap[bin] values of 1–5, the value for mantissa[bin] is determined using the code word mantissa_code[bin] as follows:

$$\text{mantissa}[\text{bin}] = \text{mantissa_value}[\text{mantissa_code}[\text{bin}]] \quad (10.43)$$

where the mantissa_value corresponding to each mantissa_code[bin] value can be found in Tables 10.25–10.29.

For bap[bin] values of 3 or 5, the value of mantissa_code[bin] is obtained directly from the corresponding 3 or 4 bits of the bit stream, respectively.

For a bap[bin] value of 1, the corresponding 5 bits of the bit stream represent a group_code value for the next three mantissa_code values to be processed that have been allocated a bap[bin] value of 1. Similarly, for a bap[bin] value of 2, the corresponding 7 bits of the bit stream represents a group_code value for three mantissa_code values, and for a bap[bin] value of 4, the corresponding 7 bits of the bit stream represents a group_code values for two mantissa_code values.

The mantissa_code values corresponding to each group_code value are extracted using the following equations:

bap = 1:

$$\begin{aligned} \text{mantissa_code}[a] &= \text{truncate}(\text{group_code}/9) \\ \text{mantissa_code}[b] &= \text{truncate}((\text{group_code}\%9)/3) \\ \text{mantissa_code}[c] &= (\text{group_code}\%9)\%3 \end{aligned} \quad (10.44)$$

bap = 2:

$$\begin{aligned} \text{mantissa_code}[a] &= \text{truncate}(\text{group_code}/25) \\ \text{mantissa_code}[b] &= \text{truncate}((\text{group_code}\%25)/5) \\ \text{mantissa_code}[c] &= (\text{group_code}\%25)\%5 \end{aligned} \quad (10.45)$$

bap = 4:

$$\begin{aligned} \text{mantissa_code}[a] &= \text{truncate}(\text{group_code}/11) \\ \text{mantissa_code}[b] &= \text{group_code}\%11 \end{aligned} \quad (10.46)$$

10.3.3.2. Asymmetric Quantization ($6 \leq \text{bap} \leq 15$)

For `bap[bin]` values of 6 to 15, the value for `mantissa[bin]` is obtained directly from the corresponding `qntztab[bap[bin]]` bits of the bit stream.

10.3.4. Decoupling

If coupling is in use, the high-frequency coefficients of the coupled channels must be reconstructed from the coupling channel coefficients and the coupling coordinates of the coupled channels. The following MATLAB code defines the algorithm used to reconstruct the coupling coordinates for each coupling band of the coupled channels from the coupling coordinate mantissa and exponent values.

```
if (cplcoexp(ch+1,bnd+1) == 15)
    cplco_temp(ch+1,bnd+1) = cplcomant(ch+1,bnd+1)/16;
else
    cplco_temp(ch+1,bnd+1) = (cplcomant(ch+1,bnd+1)+16)/32;
end
cplexp = (cplcoexp(ch+1,bnd+1)+3*mstrcplco(ch+1));
cplco(ch+1,bnd+1) = cplco_temp(ch+1,bnd+1)*(2^(-cplexp));
```

Then using the `cplbndstrc` array, the coupling coordinates for bands that span multiple coupling subbands are repeated to reconstruct the coupling coordinates for each subband.

The mantissa values for each coupled coefficient can then be determined from these coupling coordinates using the algorithm defined by the following MATLAB code.

```
for sbnd = cplbegf:cplendf+2
    for bin = 0:11
        s = sbnd*12+bin+37;
        chmant(ch+1,s+1) = cplmant(s+1)*cplco(ch+1,sbnd+1)*8;
    end
end
```

Then for each coefficient that is included in coupling, a fixed point value is obtained by right shifting this mantissa value by the corresponding exponent from the coupling channel.

10.3.5. Inverse Transform

Once the transform coefficients have been recovered from the decoded mantissa and exponent values, the inverse transform is performed to obtain the reconstructed sample values. The inverse transform is performed using the following equation:

$$y(n) = -w(n) \sum_{k=0}^{N/2-1} X(k) \cos\left(\frac{2\pi}{4N}(2n+1)(2k+1) + \frac{\pi}{4}(2k+1)(1+\alpha)\right) \quad (10.47)$$

for $n = 0, \dots, N-1$

where $X(k)$ is the set of decoded transform coefficients, $w(n)$ is the window function with the values shown in Table 10.4, and $y(n)$ is the set of windowed output samples. The variable α is used to control the time offset for the transform basis functions and is given by

$$\alpha = \begin{cases} -1, & \text{for the first short transform} \\ 0, & \text{for the long transform} \\ +1, & \text{for the second short transform} \end{cases} \quad (10.48)$$

10.3.6. Overlap and Add

The final step in the TDAC transform is to overlap the first half of the set of windowed output samples with the second half of the previous block. The following MATLAB code defines this overlap-and-add process.

```
for n=0:N/2-1
    output(n+1) = 2*(y(n+1)+delay(n+1));
    delay(n+1) = y(N/2+n+1);
end
```

10.4. SUMMARY

The ATSC digital television standard prescribes the use of an AC-3 bit stream to transmit audio information. The AC-3 audio standard defines a multichannel coding algorithm that is capable of transmitting up to five full bandwidth channels and one low-frequency effects channel in one of the eight possible channel configurations. In this chapter, the encoding algorithm, syntax, and decoding algorithm for the AC-3 standard are explained.

PROBLEMS

10.1 Draw a simple block diagram of an AC-3 encoder that includes the following system components, and briefly explain the function of each of these components:

- (a) Frequency analysis
- (b) Exponent coding

- (c) Bit allocation
 - (d) Mantissa quantization
 - (e) Bit-stream formatting
- 10.2** With the aid of a diagram describe the format of an AC-3 synchronization frame.
- 10.3** List the eight audio coding modes that are possible with an AC-3 coder and specify the order in which the channels are coded in each mode.
- 10.4** Explain what is meant by the term *preecho* in digital audio coding and describe the system used in the AC-3 coder to overcome this problem.
- 10.5** Explain how the output coefficients of the TDAC transform are converted into an exponent/mantissa representation and the reason for transmitting the coefficients in this manner.
- 10.6** Explain why channel coupling is used in the AC-3 encoder and discuss the psychoacoustic effect that allows this type of data rate reduction to be performed without introducing audible distortion in the reconstructed audio signal.
- 10.7** Explain what is meant by the following terms when used in the context of channel coding in the AC-3 encoder:
- (a) Coupling subband
 - (b) Coupling band
 - (c) Coupling coordinates
 - (d) Phase flag
 - (e) Master coupling coordinate
 - (f) Coupling channel
 - (g) Coupled channel
 - (h) Independent channel
- 10.8** Explain why rematrixing is used in the 2/0 audio coding mode.
- 10.9** Explain how the coefficient exponent values are transmitted to the decoder using differential exponents.
- 10.10** Explain the difference between the three exponent strategies D15, D25, and D45, and discuss the types of audio signals that would be best coded using each strategy.
- 10.11** Determine the absolute exponent value for the first exponent and the differential exponent values that would be transmitted to the decoder for the original exponent values given below. Assume that the D-15 exponent coding strategy is in use.
- 10.12** Determine the absolute exponent value for the first exponent and the differential exponent values that would be transmitted to the decoder for the original exponent values given below. Assume that the D-45 exponent coding strategy is in use.
- 10.13** Explain how differential exponents are transmitted to the decoder using *mapped values* and *grouped values*.
- 10.14** Complete the following table by calculating the actual bits to be transmitted to the decoder for each set of three differential exponent values.

Differential exponent values	Grouped value code word
-2,1,0	
-1,-1,-1	
1,0,-1	

- 10.15** Discuss the advantages and disadvantages of the AC-3 bit-allocation philosophy of using a standard core bit-allocation procedure that can be modified by delta bit-allocation parameters.
- 10.16** Explain what is meant by the following terms when used in the context of bit allocation in the AC-3 encoder:
- (a) PSD integration
 - (b) Bark scale band
 - (c) Log addition
 - (d) Spreading function
 - (e) Fast decay masking curve
 - (f) Slow decay masking curve
 - (g) Excitation function
- 10.17** Explain why it is necessary to compensate for the selectivity of the TDAC transform at low frequencies.
- 10.18** Explain how the masking threshold curve is calculated from the excitation function and the absolute threshold of hearing.
- 10.19** Describe the process for adjusting the masking curve produced by the core bit-allocation procedure using delta bit-allocation parameters.
- 10.20** Describe the process for determining the bit-allocation pointer values for each Bark scale band.
- 10.21** Explain the difference between the symmetric and asymmetric quantization procedures used to quantize the mantissa values.
- 10.22** Complete the following table by determining the actual bits to be transmitted to the decoder for the given mantissa and bap values.

Mantissa	bap	Code word
0.2311	3	
0.6068	5	
0.4860	7	
0.8913	9	

- 10.23** Explain the concept of dialog normalization and give examples of when it may be beneficial to use this procedure.
- 10.24** Explain the concept of dynamic range compression and give examples of when it may be beneficial to use this procedure.

10.25 Complete the following table by determining the normalized standard downmix equations that should be used in the audio reproduction system for the following input/output channel configurations:

Input channels	Output channels	Equations
L, C, R, S	L_0, R_0	
L, C, R, S	L_t, R_t	
L, C, R, S	C	
L, C, R, Ls, Rs	L_0, R_0	
L, C, R, Ls, Rs	L_t, R_t	
L, C, R, S	L, C, R, Ls, Rs	

10.26 Determine the actual bits required in the BSI header of an AC-3 syncframe for the audio stream with the following configuration:

Property	Value
Version	8
Type of service	C
Audio coding mode	1/0
Low-frequency effects channel	Off
Average dialog level	-25 dB
Heavy compression	Off
Copyright	Protected
Original	Yes
Time code (h:min:s:frame)	1:45:15:0

10.27 Determine the actual bits required in the BSI header of an AC-3 syncframe for the audio stream with the following configuration:

Property	Value
Version	8
Type of service	CM
Audio coding mode	3/2
Center mix level	-4.5 dB
Surround mix level	-6 dB
Low-frequency effects channel	on
Average dialogue level	-30 dB
Heavy compression	off
Peak mixing level	100 dB
Room type	Large room, X curve monitor
Copyright	Protected
Original	Yes
Time code (h:min:s:frame)	2:05:10:0

MATLAB EXERCISE 10.1

The aim of this exercise is to implement the spectral envelope coding and decoding algorithms used in the AC-3 encoder and decoder.

1. Write a MATLAB function to calculate the mantissa and exponent values for a set of transform coefficients using the procedure described in Section 10.1.6.
2. Write a MATLAB function to calculate the differential exponent values for a set of absolute exponent values using the procedure described in Section 10.1.7. Your function should have the exponent coding strategy as an input and be able to produce differential exponent values for the three exponent coding strategies.
3. Write a MATLAB function to convert a set of differential exponent values into mapped values and then take three mapped values and produce the bits corresponding to a grouped value.
4. Write a MATLAB function to produce a set of coefficient exponent values from a bit stream containing the code words corresponding to a set of coded group values.
5. Discuss techniques that may be used to automatically choose the most appropriate exponent coding strategy.

MATLAB EXERCISE 10.2

The aim of this exercise is to implement the bit-allocation algorithm used in the AC-3 encoder.

1. Write a MATLAB function to initialize the strtmant and endmant parameters using the code given in Section 10.1.8.1.
2. Write a MATLAB function to convert the exponent values specified by the coded spectral envelope into power spectral density values using the code given in Section 10.1.8.2.
3. Write a MATLAB function to calculate the power spectral density values for the set of 50 Bark scale bands given in Table 10.13 using the code given in Section 10.1.8.3.
4. Write a MATLAB function to calculate the excitation function values for the set of 50 Bark scale bands given in Table 10.13 using the code given in Section 10.1.8.4.
5. Write a MATLAB function to calculate the masking curve values for the set of 50 Bark scale bands given in Table 10.13, using the pseudocode given in Section 10.1.8.6.
6. Write a MATLAB function to calculate the bit-allocation pointer values for the set of transform coefficients, using the pseudocode given in Section 10.1.8.8.

7. Now combine these functions to produce a MATLAB function that determines bit-allocation pointer values for a given set of input coefficients. Determine the required `snroffset` values to produce an audio block with the following output bitrates:
 - (a) 32 kbit/s
 - (b) 128 kbit/s

MATLAB EXERCISE 10.3

The aim of this exercise is to implement the quantization and inverse quantization algorithms used in the AC-3 encoder and decoder.

1. Write a MATLAB function to quantize a set of coefficient mantissa values with the number of bits specified by a set of bit-allocation values for each coefficient using the quantization algorithm given in Section 10.1.9.
2. Write a MATLAB function to produce a set of coefficient mantissa values from a bit stream containing the code words corresponding to a set of quantized mantissa values.

REFERENCES

1. ATSC Standard: Digital Audio Compression (AC-3), Revision A, Doc. A/52A, 20 August 2001.